

DAIRY FARM MANAGEMENT SYSTEM

A Project Report

Submitted in partial fulfilment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION INFORMATION)

By

Shreekesh Yadav (8570)

Under the esteemed guidance of

Prof. Omkar Sherkhane

DEPARTMENT OF INFORMATION TECHNOLOGY



**MAHATMA EDUCATION SOCIETY'S
PILLAI COLLEGE OF ARTS, COMMERCE AND SCIENCE
(AUTONOMOUS)**

(Affiliated to University of Mumbai)

NEW PANVEL-410206, MAHARASHTRA

YEAR 2022-2023

MAHATMA EDUCATION SOCIETY'S
PILLAI COLLEGE OF ARTS, COMMERCE AND SCIENCE
(AUTONOMOUS)

(Affiliated to University of Mumbai)

DEPARTMENT OF INFORMATION TECHNOLOGY



Certificate

This is to certify that the project is entitled “**Dairy Farm Management System**”, is bona fide work of **Mr. Shreekesh Yadav** bearing Seat No: **8570** submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Date:-

Coordinator

Internal guide

College Seal

External Coordinator

ACKNOWLEDGEMENT

I, **Mr. Shreekesh Yadav** student of **Pillai College of Arts, Commerce & Science, New Panvel** would like to express my sincere gratitude towards Information Technology Department.

I would like to thank Mrs. Deepika Sharma (Vice Principal) for granting me the opportunity to build project for the college. Last but not least I thank our guide (Mr. Omkar Sherkhane) for their constant support during the project. The project would not have been completed without dedication, creativity and energy which my family provided me.

Yours faithfully,

Shreekesh Yadav
(Final Year Information Technology)

DECLARATION

I hereby declare that the project entitled, “**Dairy Farm Management System**” done at **Pillai College Of Arts, Commerce and Science(Autonomous), New Panvel**, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATIONTECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

Name And Signature of the Student

Mr. Shreekesh Yadav

TABLE OF CONTENTS

	INDEX	
Sr. No	Chapter 1	Page No.
1	Introduction	7-10
1.1	Introduction	
1.2	Abstract	
1.3	Objective	
1.4	Scope	
	Chapter 2	11-18
2	System Planning	
2.1	Survey Of Technologies	
2.2	Fact Finding	
2.3	Feasibility Study	
2.4	Stakeholders	
	Chapter 3	19-33
3	Requirements and Analysis	
3.1	Problem Definition	
3.2	Requirement Specification	
3.3	Planning and Scheduling	
3.4	Software and Hardware Requirements	
3.5	Conceptual Diagrams	
3.5.1	Flowchart	
3.5.2	ER Diagram	
3.5.3	Use Case Diagram	
3.5.4	Class Diagram	
3.5.5	Component Diagram	
	Chapter 4	34-44
4	System Design	
4.1	Data Design	
4.2	Data Integrity and Constraints	
4.3	User Interface Design	
4.4	Test Cases	
	Chapter 5	45-77
5	System Coding, Implementation and Testing	

5.1	Coding Details and Efficiency	
5.2	Testing Approach	
5.2.1	Unit Testing	
5.2.2	Integrated Testing	
5.3	Modification and Improvement	
	Chapter 6	78
6	Conclusion	
	Chapter 7	79
7	Reference	

Chapter 1

Introduction

1.1 INTRODUCTION

Dairy farming has been an important part of the agricultural scenario for thousands of years. India being a predominantly agrarian economy has about 70 per cent of its population living in villages, where livestock play a crucial role in the socio-economic life. Livestock provide high-quality foods such as milk, cheese, butter, ghee, etc. India is not only one of the top producers of milk in the world, but also the largest consumer of milk and milk products in the world. Due to the shortfall in supply, we have to import significant amounts of milk products to meet internal demand.

Agriculture and animal husbandry have a symbiotic relationship, in which the agricultural sector provides feed and fodder for the livestock and animals provide milk, manure and draught power for various agricultural operations. Dairy sector is instrumental in bringing socio-economic transformation in India. It has created a lot of employment opportunities and also provides improved nutritional benefits.

Animal husbandry is a major sub-sector of agriculture sector and contributes about 28.6 per cent to the agricultural value output (Annual Report 2017–18, Department of Animal Husbandry, Dairying and Fisheries, Government of India). The growth rate of 6.27 per cent during 2015–16 and 2016–17, respectively in the livestock sub-sector is much faster than the overall 0.7 per cent and 4.9 per cent growth rate of agriculture and allied sector during the same period (Economic Survey, 2016–17). The statistics indicate that there is a lot of employment potential for entrepreneurial activities in the dairy sector.

1.2 ABSTRACT

The capstone project, “Dairy Farm Management System,” intends to develop an online platform for dairy farm management. The system is intended for farm owners and managers that want to use a single system to manage their whole farm operations. In recent years, dairy farms have been handled more than any other animal. The number of dairy cows that provide milk to society has been increasing recently, and management has found it difficult to manage this operation because it involves many transactions, such as vaccinations, feed monitoring, milk and meat sales, and cow information, and handling it manually results in a lot of cost and effort. In this method, the researcher provides a more appropriate response that is directed online to keep the work under control. The researcher came up with the notion of conducting the capstone project to address the concerns described above. The researchers will construct a project using the Software Development Life Cycle (SDLC) method. The researcher will test and check the app after it has been developed to confirm its functionality and seamless operation. A panel of IT specialists will review the developed application for future enhancement and improvement. The system’s target users are individuals looking for a rapid, efficient, and secure online-based dairy farm management platform.

1.3 OBJECTIVE

- ❖ To build a system that helps owner and employees of dairy farm to keep and maintain records.
- ❖ To keep record of farms growth.
- ❖ To make any research easier with the help of the application.
- ❖ To maintain the health history of cows and calves.

The Existing Dairy Farm doesn't use any Application or System for keeping records which makes hard for some to check the records and do research whereas this application will help to keep records for longer time then the hard copy. It is easy to take backup and create new records.

1.4 SCOPE

This Application has huge scope as it makes several difficult tasks of owner and employees of Dairy Farm. It helps to keep record of milk sales, cow's health, breeding, finance as well as Employees record without any difficulty and it is easy to use for anyone with the help of system's graphical user interface which uses several logo that makes user to operate the application.

Chapter 2

System Planning

2.1 SURVEY OF TECHNOLOGIES

A mixed method approach was used to gather experiences specifically related to precision technologies from farms who were also involved in a Dairy NZ study, which focussed on milking efficiency (Edwards et al. 2012). A survey was developed to examine issues related to use of PD technologies. The survey design was based on findings from previous PD studies (Yule and Eastwood 2011; Eastwood et al. 2012) with questions based around the themes of: reasons for investment in technology, initial expectations of benefits, routines and maintenance planning, the impact of technology and perceived benefits. The survey was conducted as part of four workshops held in 2011 located in the Waikato, Taranaki, Canterbury and Southland regions in New Zealand. Representatives from the 60 farms involved in the Dairy NZ milking efficiency study (Edwards et al. 2012) were invited to attend the workshops to discuss their use of technologies. Those farmers who were not present at the workshops were subsequently surveyed by mail. Throughout the survey, respondents were asked to rate a series of statements. Most questions used a 5-point scale (1 representing 'strongly disagree' and 5 'strongly agree' whereas an answer of 3 indicated a neutral response), one question used a 5-point scale from 'Not useful' to 'Highly useful', and another used a 4-point scale from 'none' to 'a lot'. Statistical analyses Data were collated and analysed using Excel. Additional farmer comments were collected during discussion sessions at each of the four workshops where participants were surveyed about changes that have taken place on their farms as a result of using this technology. Data are primarily presented as counts and percentage responses. The 5-point scale data are collated as agree (strongly agree and agree), neutral, and disagree (strongly disagree and disagree) in the Results section. These data are reported rather than means due to the potential impact on means from skewed or divergent responses to individual questions (Clason and Dormody 1994).

New Technologies in dairy farm:

❖ Fitness tracker for cattle

Cattle productivity, lifespan, and reproductivity are all reduced by health problems. Farmers spend a lot of money on the health and well-being of their cattle every year. Farmers, on the other hand, may track, monitor, and control nutrition, behavior, pregnancy, milking frequency, milk output anomaly, and activity level in real-time owing to wearable animal gadgets similar to human fitness trackers. These intelligent animal trackers may be placed in ears, tails, legs, neck, or any other portion of the cattle body. The Karnataka government implanted GPS-enabled digital chips in the ears of 56 lakh animals around the state to track their wellbeing and detect medical conditions early. SmaXtec, Cowlar, Moocall, Smartbow, Stellapps, and other firms have created smart cow health tracking devices.

Milking via Robots:

Cows have traditionally been milked manually. This is not only a time-consuming operation, but it also has a labour cost, which raises the price of milk. Robotic milking allow farmers to minimize the need for physical labour, maintain a sanitary milking procedure, milk cows at any time of day rather than on a set schedule and increase milk output. The robotic milking equipment has arms or cups with sensors that may be fitted to the teats of cows individually. The sensors can detect whether or not the cow or one of its teats is ready for milking. Once the milking process begins, the devices can detect contaminants, color, and [milk quality](#). If the milk cannot be consumed by humans, it is directed to a different container. When the process is completed, the devices may also clean and disinfect the teats automatically. Raghava Gowda of India invented a non-electric milking mechanism for cows. Among the numerous other startups that have launched automated milking systems are miRobot, GEA, DeLaval, Fullwood Packo, and Lely.

Drones for Cattle Monitoring:

Farmers are required to maintain manual vigilance anytime cattle leave the farm for grazing. There is a considerable possibility that the cattle may become lost, stolen, or attacked by other animals. Cattle monitoring drones can track the cattle and herd them back from the fields to the barns. Some drones are outfitted with thermal sensing equipment, which allows them to track cattle based on their body heat. Drones may also scan pasture lands and communicate information about whether they are suitable for cattle grazing TRITHI Robotics, Dronitech, Sagar Defence Engineering, DJI Enterprise, and Sunbirds are among the companies that have made progress in developing commercial drones for a variety of applications, including agricultural and livestock management.

2.2 FACT FINDING

Fact finding is process of collection of data and information based on techniques which contain sampling of existing documents, research, observation, questionnaires, interviews, prototyping and joint requirements planning. System analyst uses suitable fact-finding techniques to develop and implement the current existing system. Collecting required facts are very important to apply tools in System Development Life Cycle (SDLC) because tools cannot be used efficiently and effectively without proper extracting from facts. Fact-finding techniques are used in the early stage of System Development Life Cycle including system analysis phase, design and post implementation review. Facts included in any information system can be tested based on three steps: data- facts used to create useful information, process- functions to perform the objectives and interface- designs to interact with users.

2.3 FEASIBILITY STUDY

What Is Feasibility

"Analysis of a project's costs & revenues to determine whether or not it is logical or possible to complete".

Crucial Factors In Decision Making Before Investment

- **STRENGTHS:-**
- Major source of food, i.e. milk & meat.
- Favourable breeding backgrounds.
- Relatively cheap farmland.
- High domestic consumption.
- Full family involvement.

Weaknesses

- High risk of diseases in Livestock.
- Poor management in many cases.
- Lack of education in farmers.
- Unaware of basic farm management practices.
- No or low application of research work & pedigree record keeping.

Market Entry Timing

- Milk demand is higher in summer, therefore animals are sold at higher rates.
- So the business can be started before the onset of summer.

Herd Size

- Initial herd size should be 25 animals.
- The farm size will increase to 125 animals within 10 years.

Project Investment

- Total cost of project is Rs.5,189,472.Out of which Capital cost of project is 4,928,874, rest is used to meet the working capital requirement.

Account Head	Total Cost
Land	500,000
Building/infrastructure	1,592,874
Machinery & equipment	275,000
Animal Cost	2,450,000
Pre-operating Cost	111,000
TOTAL CAPITAL COST	4,928,874

Farm Inputs

- **LAND**
- About 1 acre of land will be purchased for 25-125 animals.
- Land cost per acre is at least 5-6 lacs at neighboring rural areas.

Land requirements

- (For Target size of around 125 animals)

DESCRIPTION	AREA (Acres)
Shed for Cows	0.16
Open Paddock for Cows	0.16
Shed for Buffaloes	0.06
Open Paddock for Buffaloes	0.06
Shed for Calves	0.07
Open Paddock for Calves	0.07
Shed for Calves (Older than 1 year)	0.06
Open Paddock for calves (older	0.06

Animal Housing

- Sheds must be;
- Airy
- Facilitated with drinking water &
- Proper drainage system

Total Infrastructure Cost

DESCRIPTION	Sq.ft	Rate/Sq.ft	Total Cost
Shed for Cows	2,300	300	690,000
Open Paddock for Cows	2,300	10	23,000
Shed for Buffaloes	500	300	150,000
Open paddock for Buffaloes	500	10	5,000
Shed for Calves	692	300	207,600
Open paddock for Calves	692	10	6,920
Sheds for Calves (older than 1 year)	463	300	138,915
Open paddock for Calves (older than 1 year)	463	10	4,631
Stores for feed & machine room	150	300	45,000
Utensils & milk storage	150	300	45,000
Servant room, wash room	268	400	107,153

Herd Mix

- Ideal mixed herd should consist of 80% Cows & 20% buffalos.
- **Species**
- **Buffaloes:-** Nilli Ravi , Kundi
- **Cattle:-** Sahiwal , Red Sindhi
- **Cross Breed:-** Swl x jersey

Feed

- Concentrates, dry & green roughages are given to the animals as per their requirement.

Vaccination & Medication

- Each animal will be vaccinated before entering to the farm.
- Cost of vaccine

Labor Requirement

- One mature person can handle 15 milk animals easily.
- 3 farm workers are recommended for handling 25 animals.
- A farm manager can be hired with B.Sc degree in animal husbandry.

2.4 STAKE HOLDERS

Stakeholders are individuals who either care about or have a vested interest in your project. They are the people who are actively involved with the work of the project or have something to either gain or lose as a result of the project.

Project Stakeholders

Top Management

Top management may include the president of the company, vice-presidents, directors, division managers, the corporate operating committee, and others. These people direct the strategy and development of the organization.

Developers

The developers are the ones who design, test and maintain the project.

Users

User are those group of people who will use the website.

Investor

Investors are those who invest their money in our website for their profits.

Advertiser

These are the group of people who promotes their products or services in our website.

Chapter 3

Requirement and Analysis

3.1 PROBLEM DEFINITION

The Existing system being manual is a slow and moreover it's very difficult to access the reports of connections and transactions. Thus, for accurate and fast retrieval of reports the system was to be automated.

Disadvantages of the existing system

- ❖ No information regarding any products to the customer.
- ❖ Complicated to manage the payment and customer details.
- ❖ At present there is a manual system used to maintain all customer details.
- ❖ At present it is difficult to keep records of income and expenditure.

3.2 REQUIREMENT SPECIFICATIONS

Proposed system will automate all the work done manually in existing Dairy Farm. It will store all the records of Farm using the system and employee can maintain all the records and details easily.

“Dairy Farm Management System” will manage all the manual work of any dairy farm. Employees can enter cows details, milk production, health report, breeding, milk sales and finance.

Advantages:

- ❖ Our system is easy to use and operate.
- ❖ Easy to maintain records.
- ❖ Report generation is easy.

3.3 PLANNING AND SCHEDULING

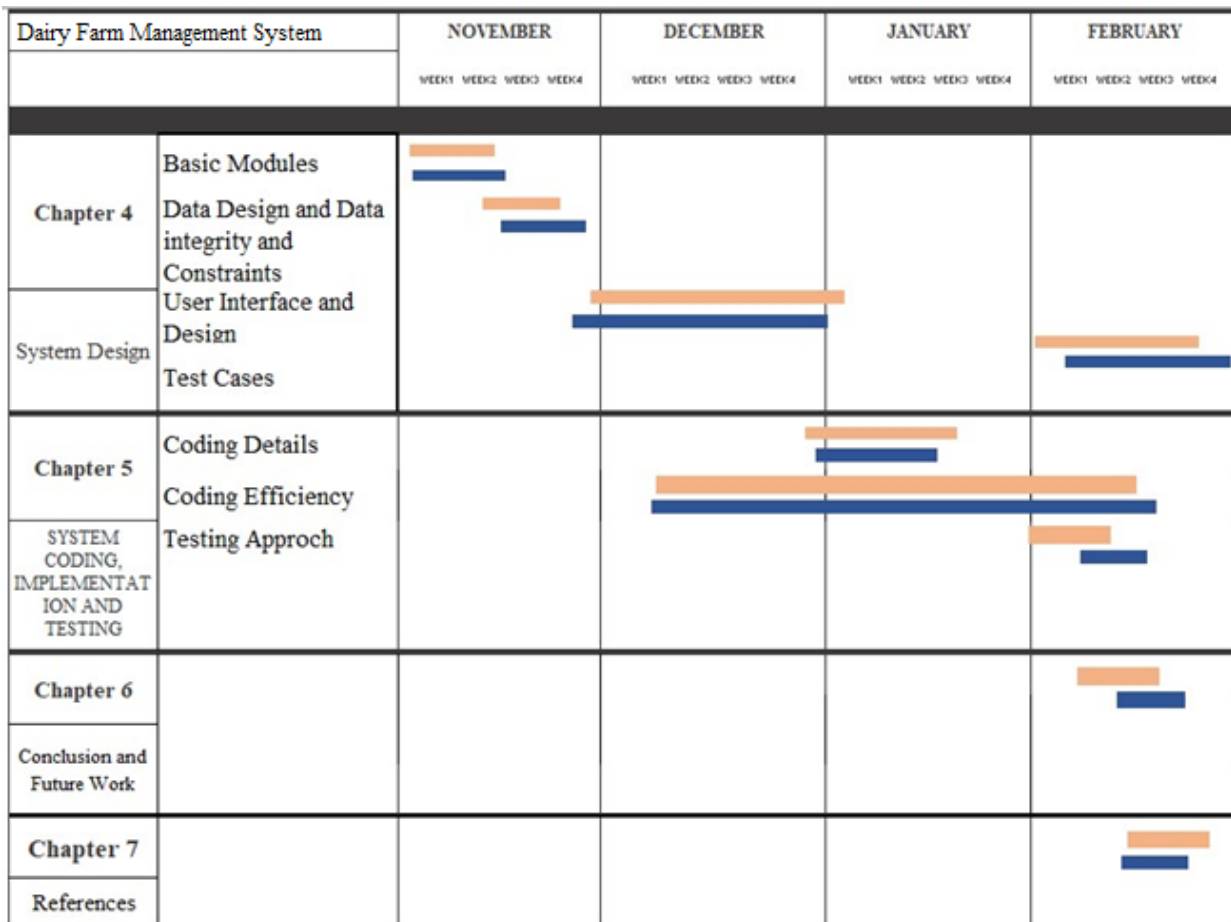
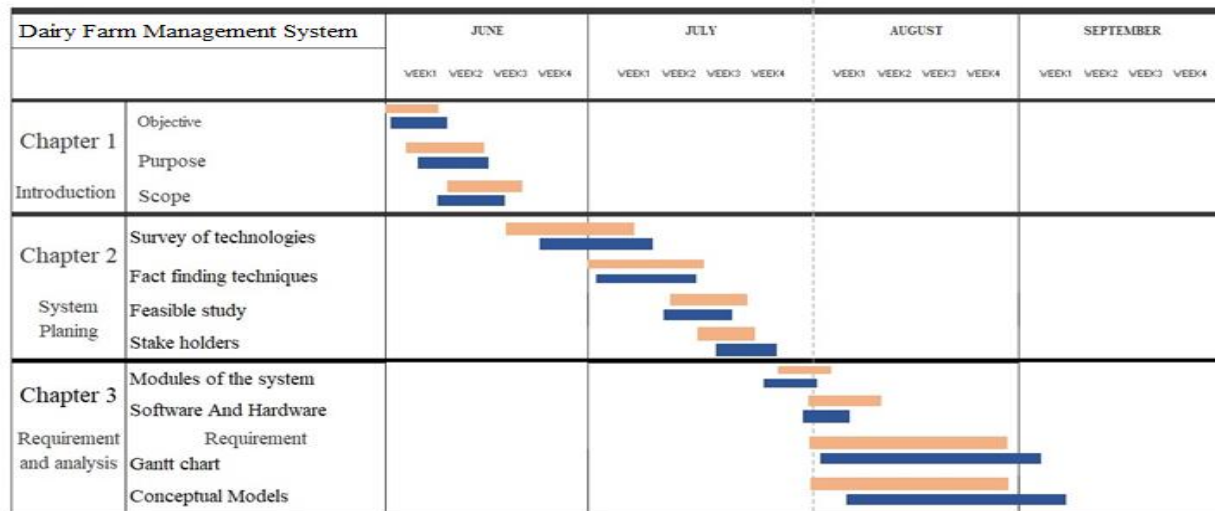
Planning and scheduling are a complicated part of software development. Planning, for our purposes, can be thought of as determining all the small tasks that must be carried out in order to accomplish the goal. Planning also takes into account, rules, known as constraints, which, control when certain tasks can or cannot happen. Scheduling can be thought of as determining whether adequate resources are available to carry out the plan. Purpose of planning and scheduling is that to finish work properly in the given time period.

For our project, planning was started from the month of June. In the planning process, we have discussed about the project purpose, reason of the existence, comparison with the existing work and the new thing to be developed in the beginning part. In the later section, after having the clear vision of the project, we started working on documentation. During the planning part, we have also discussed about the software and hardware requirement and the languages required for coding.

After all the planning part is done, according to it scheduling of work was started. In this part, time and resources are distributed according to our requirement. Purpose of scheduling is that, the work should be done without any obstacles for example, overlapping two works at the same time or same resource shouldn't be used at two different places at the same time. To avoid this circumstance, planning and scheduling is done.

Planning and scheduling can be shown with the help of using Gantt Chart. It properly shows when the work starts, how much time is required and who is doing which work. It gives us the overview knowledge of time duration and working structure of the project. It even helps us to know if the work is going in the estimated time duration or not.

Gantt Chart



3.4 SOFTWARE AND HARDWARE REQUIREMENT

3.4.1 Hardware Requirement

- PROCESSOR: 1.8 GHz or faster processor
- RAM: 4GB
- DISK SPACE: 256GB

3.4.2 Software Requirement

- Application: Visual Studio Community 2019 or higher
- Operating System: Windows/ Linux/ Mac OS
- Database: MySql

3.5 CONCEPTUAL MODELS

A conceptual model is a representation of a system, made of the composition of concepts which are used to help people know, understand, or simulate a subject the model represents. It is also a set of concepts. Conceptual modelling is a crucial software development activity.

Conceptual models give you background details about the project work. It tells you about the resources, criteria, requirements, involvements of people, process, etc.

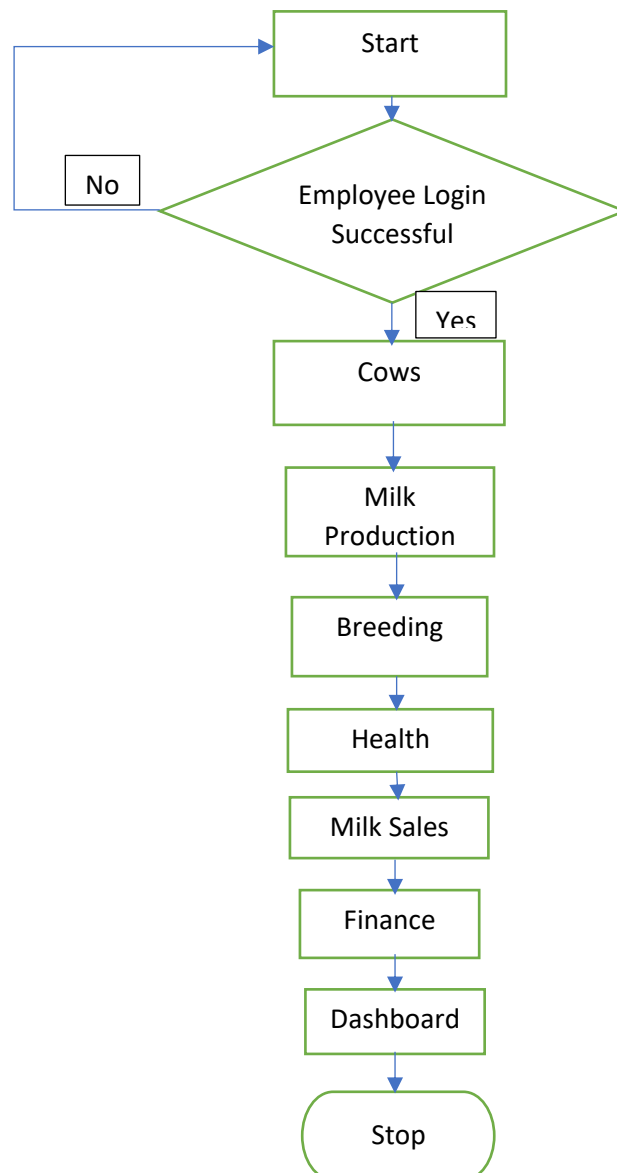
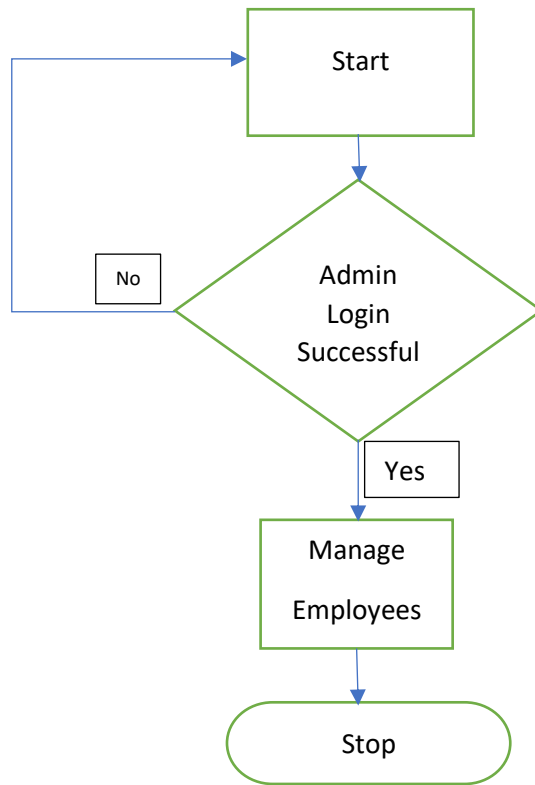
3.5.1 FLOWCHART

A flow chart is a graphical or symbolic representation of a process. Each step in the process is represented by a different symbol and contains a short description of the process step. The flow chart symbols are linked together with arrows showing the process flow direction.

Different flow chart symbols have different meanings. The most common flow chart symbols are:

- Terminator: An oval flow chart shape indicating the start or end of the process.
- Process: A rectangular flow chart shape indicating a normal process flow step.
- Decision: A diamond flow chart shape indication a branch in the process flow.
- Connector: A small, labelled, circular flow chart shape used to indicate a jump in the process flow. (Shown as the circle with the letter "A", below.)
- Data: A parallelogram that indicates data input or output (I/O) for a process.
- Document: Used to indicate a document or report (see image in sample flow chart below).

A simple flow chart showing the symbols described above can be seen below:



3.5.2 ER DIAGRAM

The Entity-Relationship (ER) model was originally proposed by Peter in 1976 as a way to unify the network and relational database views. Simply stated the ER model is a conceptual data model that views the real world as entities and relationships. A basic component of the model is the Entity Relationship diagram which is used to visually represent data objects. Since Chen wrote his paper the model has been extended and today it is commonly used for database design for the database designer, the utility of the ER model is:

- It maps well to the relational model. The constructs used in the ER model can easily be transformed into relational tables.
- It is simple and easy to understand with a minimum of training. Therefore, the model can be used by the database designer to communicate the design to the end user.

In addition, the model can be used as a design plan by the database developer to implement a data model in specific database management software.

Connectivity and Cardinality

The basic types of connectivity for relations are: one-to-one, one-to-many, and many-to-many.

- A one-to-one (1:1) relationship is when at most one instance of a entity A is associated with one instance of entity B.

For example, "employees in the company are each assigned their own office. For each employee there exists a unique office and for each office there exists a unique employee.

- A one-to-many (1:N) relationships is when for one instance of entity A, there are zero, one, or many instances of entity B, but for one instance of entity B, there is only one instance of entity A.

An example of a 1:N relationships is a department has many employees each employee is assigned to one department

- A many-to-many (M:N) relationship, sometimes called non-specific, is when for one instance of entity A, there are zero, one, or many instances of entity B and for one instance of entity B there are zero, one, or many instances of entity A. The connectivity of a relationship describes the mapping of associated.

ER Notation

There is no standard for representing data objects in ER diagrams. Each modeling methodology uses its own notation. The original notation used by Chen is widely used in academics texts and journals but rarely seen in either CASE tools or publications by non-academics. Today, there are a number of notations used; among the more common are Bachman, crow's foot, and IDEFIX.

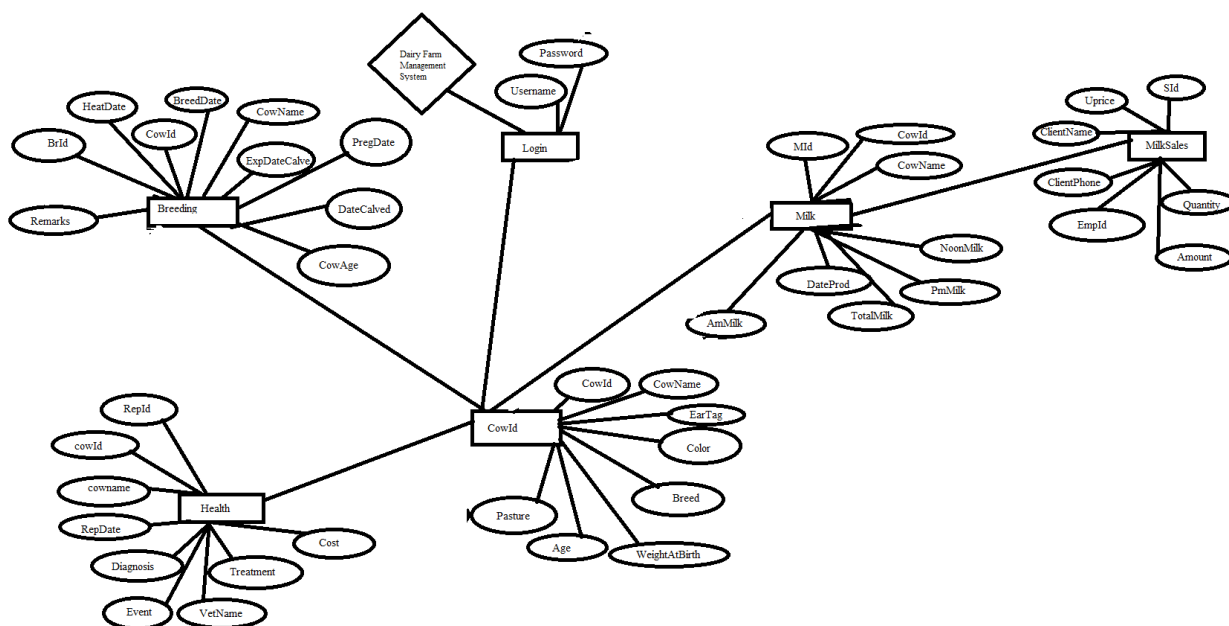
All notational styles represent entities as rectangular boxes and relationships as lines connecting boxes. Each style uses a special set of symbols to represent the cardinality of a

connection. The notation used in this document is from Martin. The symbols used for the basic ER constructs are:

- Entities are represented by labeled rectangles. The label is the name of the entity. Entity names should be singular nouns.
- Relationships are represented by a solid line connecting two entities. The name of the relationship is written above the line. Relationship names should be verbs
- Attributes when included, are listed inside the entity rectangle. Attributes which are identifiers are underlined. Attribute names should be singular nouns.
- Cardinality of many is represented by a line ending in a crow's foot. If the crow's foot is omitted, the cardinality is one.

Existence is represented by placing a circle or a perpendicular bar on the line. Mandatory existence is shown by the bar (looks like a 1) next to the entity for an instance is required.

Optional existence is shown by placing a circle next to the entity that is option



3.5.3 USE CASE DIAGRAM:

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use case in which the user is involved. A use case diagram can identify the different types of users of a system and the different use cases and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses.

Symbol used for use case are as follows



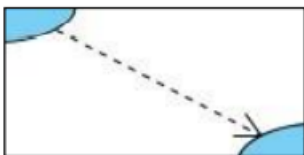
Represents users of system.



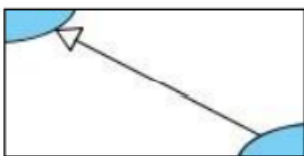
Represents a user goal.



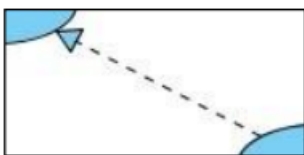
Use case are placed in the system.



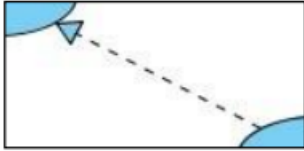
Represents the connection between Entity.



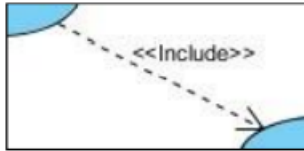
Represents a model element relies on another model element.



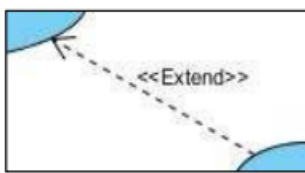
Represent inheritance relationship between model elements of same type.



Relationship between a specification and its implementation.



A use case contains the behavior defined in another use case.

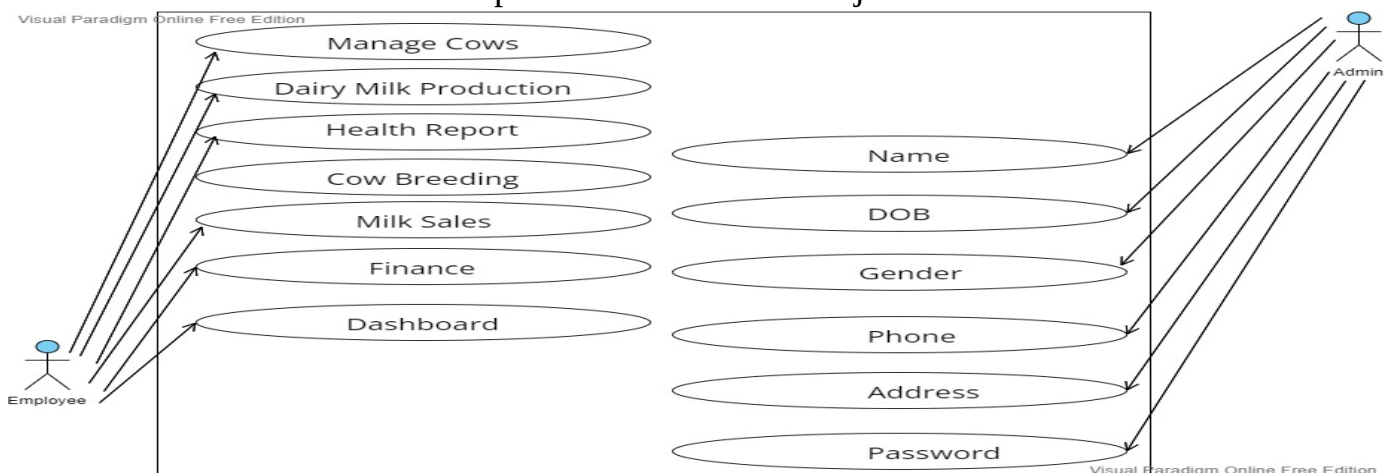


Extend is intended to be used when there is some additional behavior that should be added

Use case diagrams model behavior within a system and helps the developers understand of what the user requires. The stick man represents what's called an actor. Use case diagram can be useful for getting an overall view of the system and clarifying that can do and more importantly what they can't do. Use case diagram consists of use cases and actors and shows the interaction between the use case and actors.

- The purpose is to show the interactions between the use case and actor.
- To represent the system requirements from user's perspective.
- An actor could be the end-user of the system or an external system.

A Use case is a description of set of sequence of actions graphically it is rendered as an ellipse with solid line including only its name. Use case diagram is a behavioral diagram that shows a set of use cases and actors and their relationship. It is an association between the use cases and actors. An actor represents a real-world object.



3.5.4 CLASS DIAGRAM

In software engineering, a class diagram in the (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

The class diagram is the main building block of object-oriented modelling. It is used for general conceptual modelling of the structure of the application, and for detailed modelling translating the models into programming code. Class diagrams can also be used for data modelling. The classes in a class diagram represent both the main elements, interactions in the application, and the classes to be programmed.

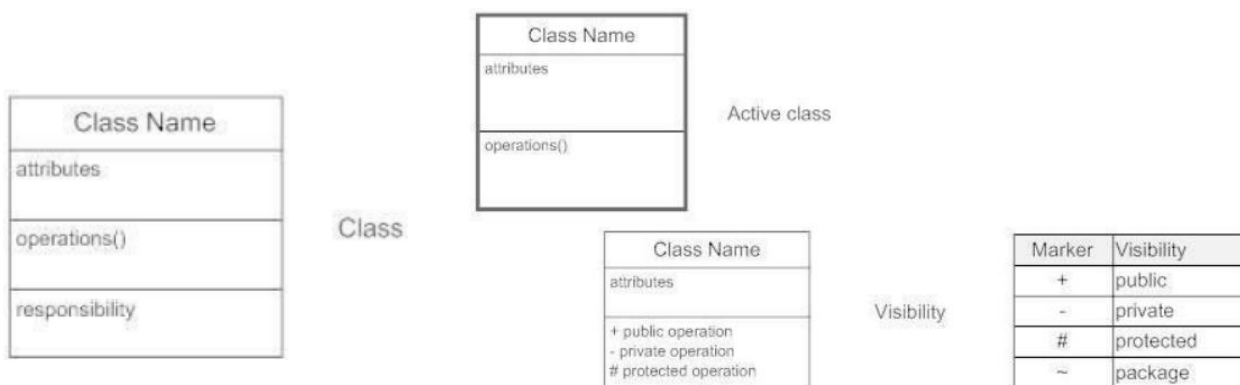
In the diagram, classes are represented with boxes that contain three compartments:

- The top compartment contains the name of the class. It is printed in bold and centered, and the first letter is capitalized.
- The middle compartment contains the attributes of the class. They are left-aligned and the first letter is lowercase.
- The bottom compartment contains the operations the class can execute. They are also left-aligned and the first letter is lowercase.

Basic Class Diagram Symbols and Notations Classes

Classes represent an abstraction of entities with common characteristics. Associations represent the relationships between classes.

Illustrate classes with rectangles divided into compartments. Place the name of the class in the first partition (centered, bolded, and capitalized), list the attributes in the second partition (left-aligned, not bolded, and lowercase), and write operations into the third.

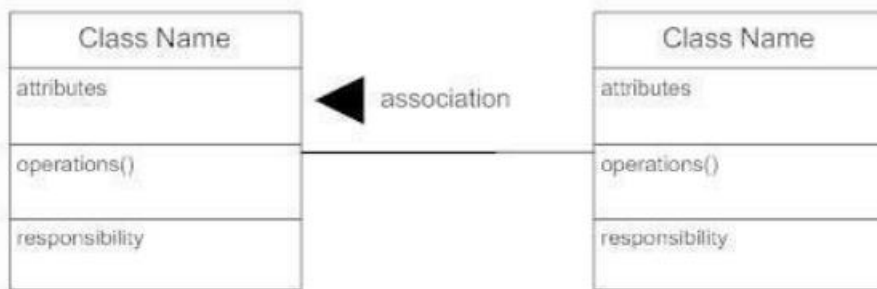


Use visibility markers to signify who can access the information contained within a class. Private visibility, denoted with a - sign, hides information from anything outside the class partition.

Public visibility, denoted with a + sign, allows all other classes to view the marked information.

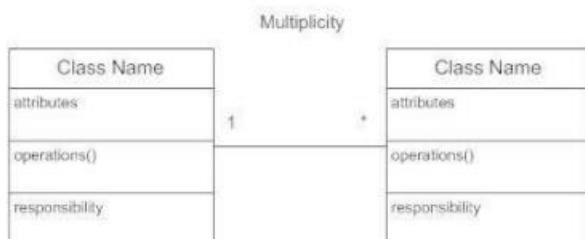
Protected visibility, denoted with a # sign, allows child classes to access information they inherited from a parent class.

Associations



Associations represent static relationships between classes. Place association names above, on, or below the association line. Use a filled arrow to indicate the direction of the relationship. Place roles near the end of an association. Roles represent the way the two classes see each other.

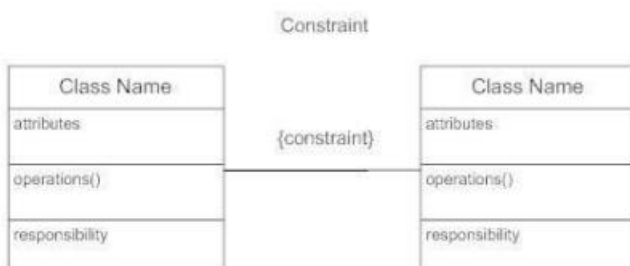
Multiplicity (Cardinality) Place multiplicity notations near the ends of an association. These symbols indicate the number of instances of one class linked to one instance of the other class. For example, one company will have one or more employees, but each employee works for just one company.



Indicator	Meaning
0..1	Zero or one
1	One only
0..*	0 or more
1..*	1 or more
n	Only n (where n > 1)
0..n	Zero to n (where n > 1)
1..n	One to n (where n > 1)

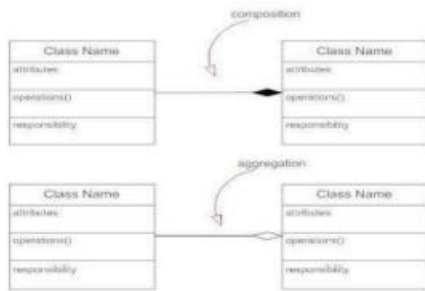
Constraint

Place constraints inside curly braces {}.



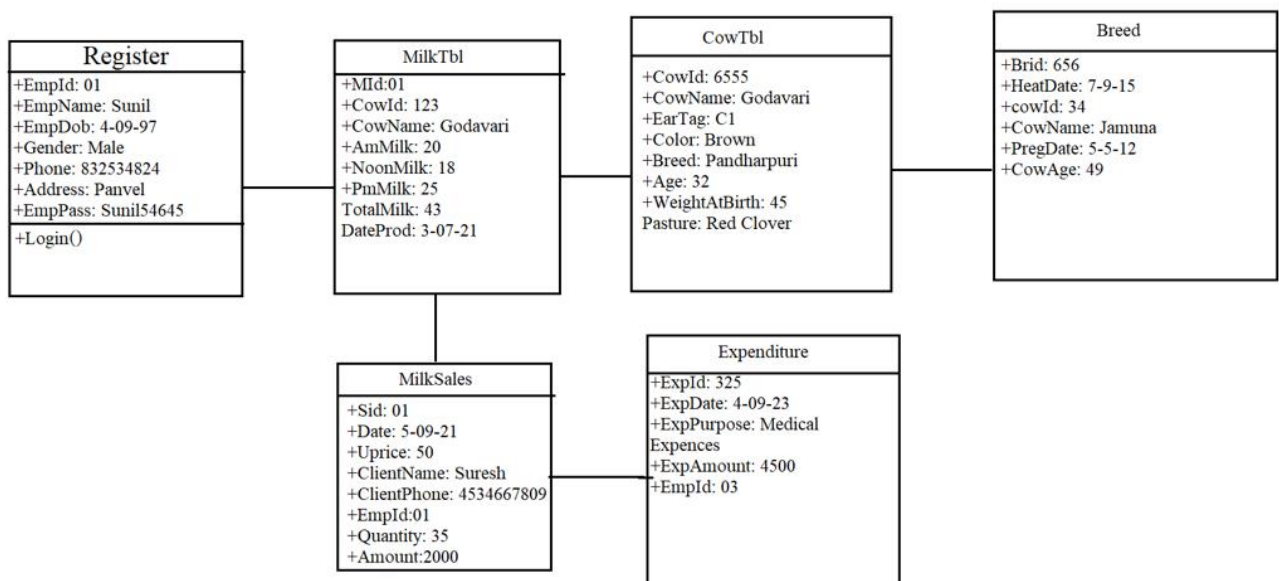
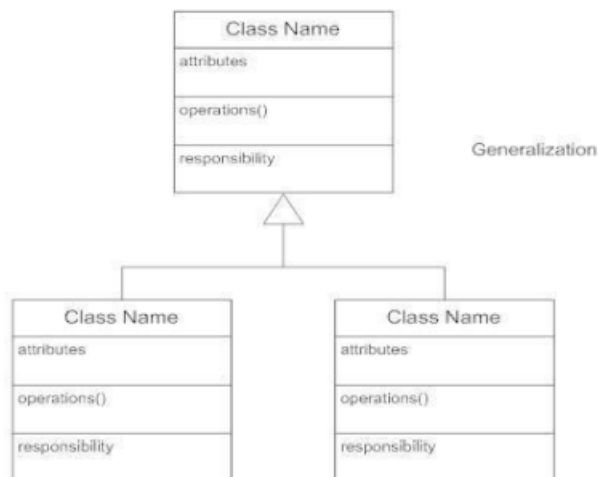
Composition and Aggregation Composition is a special type of aggregation that denotes a strong ownership between Class A, the whole, and Class B, its part. Illustrate composition with a filled diamond. Use a hollow diamond to represent a simple aggregation relationship, in which the "whole" class plays a more important role than the "part" class, but the two

classes are not dependent on each other. The diamond ends in both composition and aggregation relationships point toward the "whole" class (i.e., the aggregation).



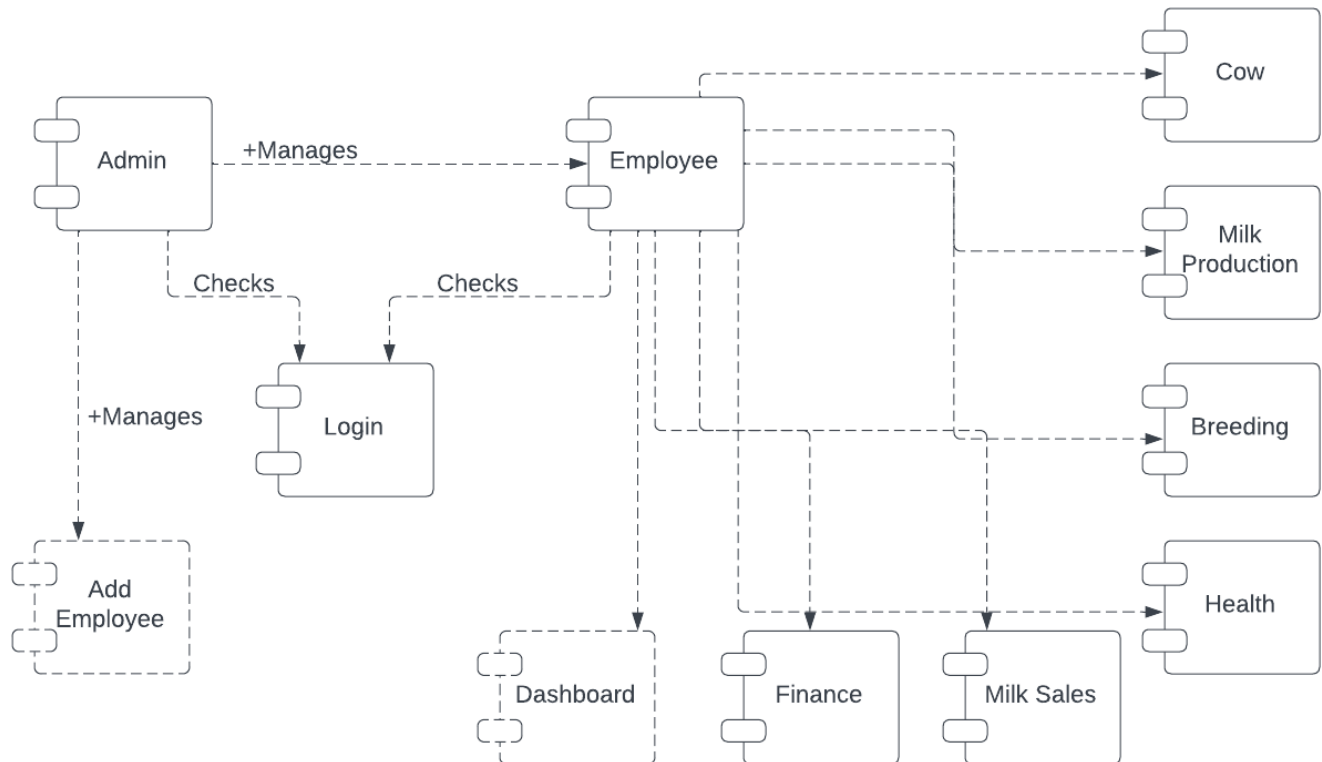
Generalization

Generalization is another name for inheritance or an "is a" relationship. It refers to a relationship between two classes where one class is a specialized version of another. For example, Honda is a type of car. So, the class Honda would have a generalization relationship with the class car.



3.5.5 COMPONENT DIAGRAM

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and doublecheck that every aspect of the system's required function's is covered by planned development.

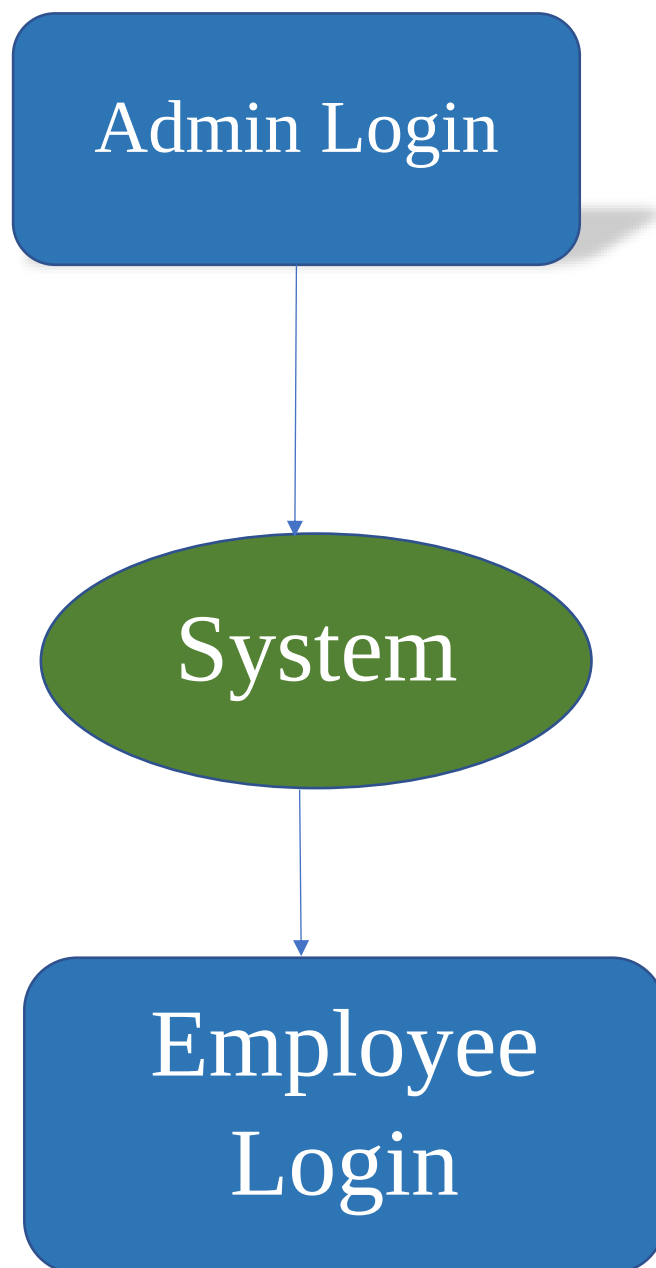


Chapter 4

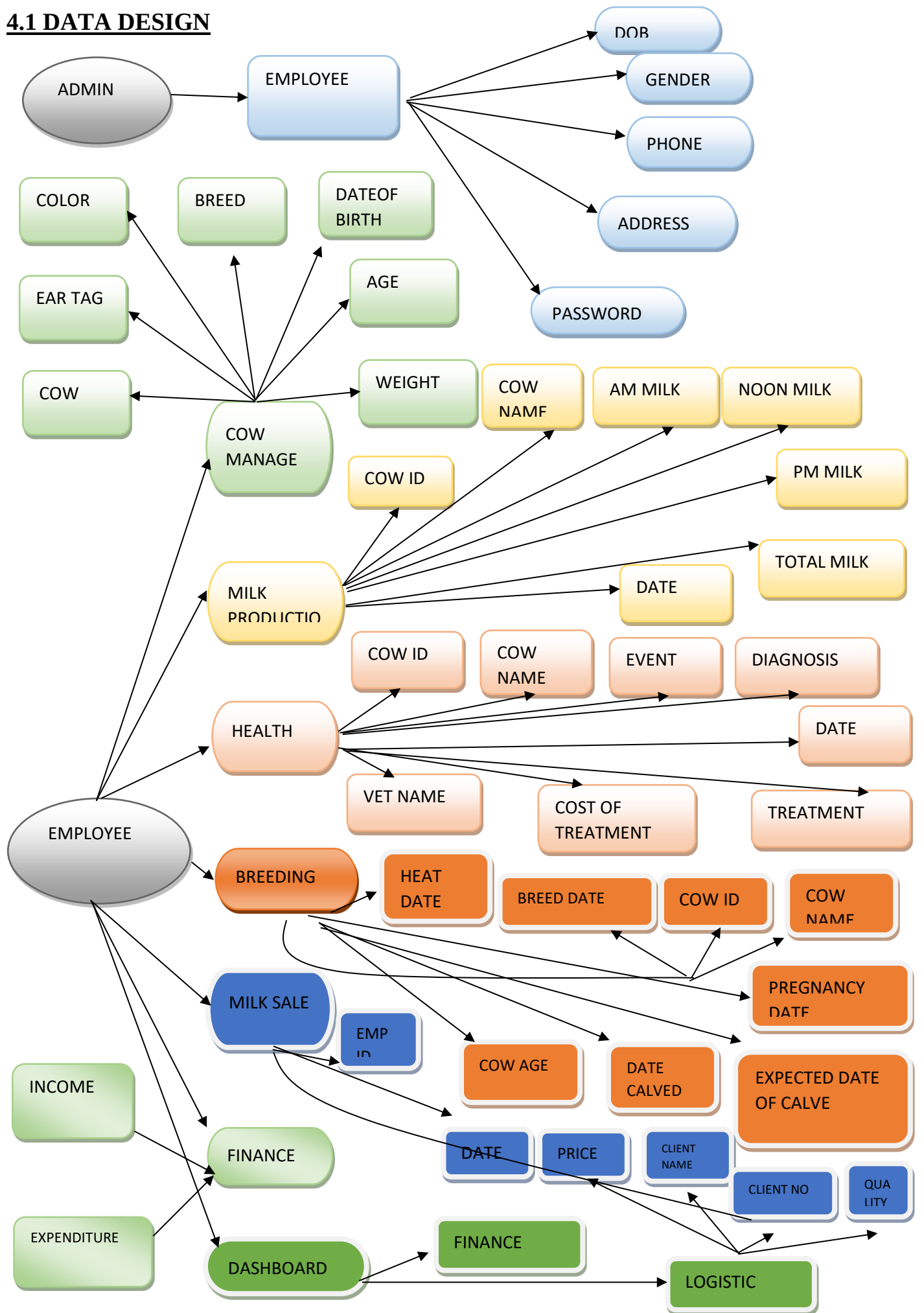
SYSTEM DESIGN

System Design is the process of designing the architecture, components, and interfaces for a system so that it meets the end-user requirements.

It's a wide field of study in Engineering and includes various concepts and principles that will help you in designing scalable systems. These concepts are extensively asked in the Interview Rounds for SDE 2 and SDE 3 Positions at various tech companies. These senior roles demand a better understanding of how you solve a particular design problem, how you respond when there is more than expected traffic on your system, how you design the database of your system and many more. All these decisions are required to be taken carefully keeping in mind Scalability, Reliability, Availability, and Maintainability. We will be covering all of these terminologies in this article.



4.1 DATA DESIGN



4.2 DATA INTEGRITY AND CONSTRAINTS

BREED TABLE

ATTRIBUTES	DATA TYPE	CONSTRAINTS
BrId	int	Primary Key
HeatDate	date	Not Null
BreedDate	date	Not Null
CowId	int	Not Null
CowName	varchar(50)	Not Null
PregDate	date	Not Null
ExpDateCalve	date	Not Null
DateCalved	date	Not Null
Cow Age	int	Not Null
Remarks	varchar(50)	Not Null

COW TABLE

ATTRIBUTES	DATA TYPE	CONSTRAINTS
CowId	int	Primary Key
CowName	varchar(50)	Not Null
EarTag	varchar(50)	Not Null
Color	varchar(50)	Not Null
Breed	varchar(50)	Not Null
Age	int	Not Null
WeightAtBirth	int	Not Null
Pasture	varchar(50)	Not Null

EMPLOYEE TABLE

AATTRIBUTES	DATA TYPE	CONSTRAINTS
EmpId	int	Primary Key
EmpName	varchar(50)	Not Null
EmpDob	date	Not Null
Gender	varchar(50)	Not Null
Phone	varchar(50)	Not Null
Address	varchar(50)	Not Null
EmpPass	varchar(50)	Not Null

EXPENDITURE TABLE

ATTRIBUTES	DATA TYPE	CONSTRAINTS
EmpId	int	Primary Key
ExpDate	date	Not Null
ExpPurpose	varchar(50)	Not Null
ExpAmount	int	Not Null
EmpId	int	Not Null

HEALTH TABLE

ATTRIBUTES	DATA TYPE	CONSTRAINTS
RepId	int	Primary Key
cowId	int	Not Null
Cowname	varchar(50)	Not Null
RepDate	date	Not Null
Event	varchar(50)	Not Null
Diagnosis	varchar(50)	Not Null
Treatment	varchar(50)	Not Null
Cost	int	Not Null
VetName	varchar(50)	Not Null

INCOME TABLE

ATTRIBUTES	DATA TYPE	CONSTRAINTS
IncId	int	Primary Key
IncDate	date	Not Null
IncPurpose	varchar(50)	Not Null
IncAmt	int	Not Null
EmpId	int	Not Null

MILKSALES TABLE

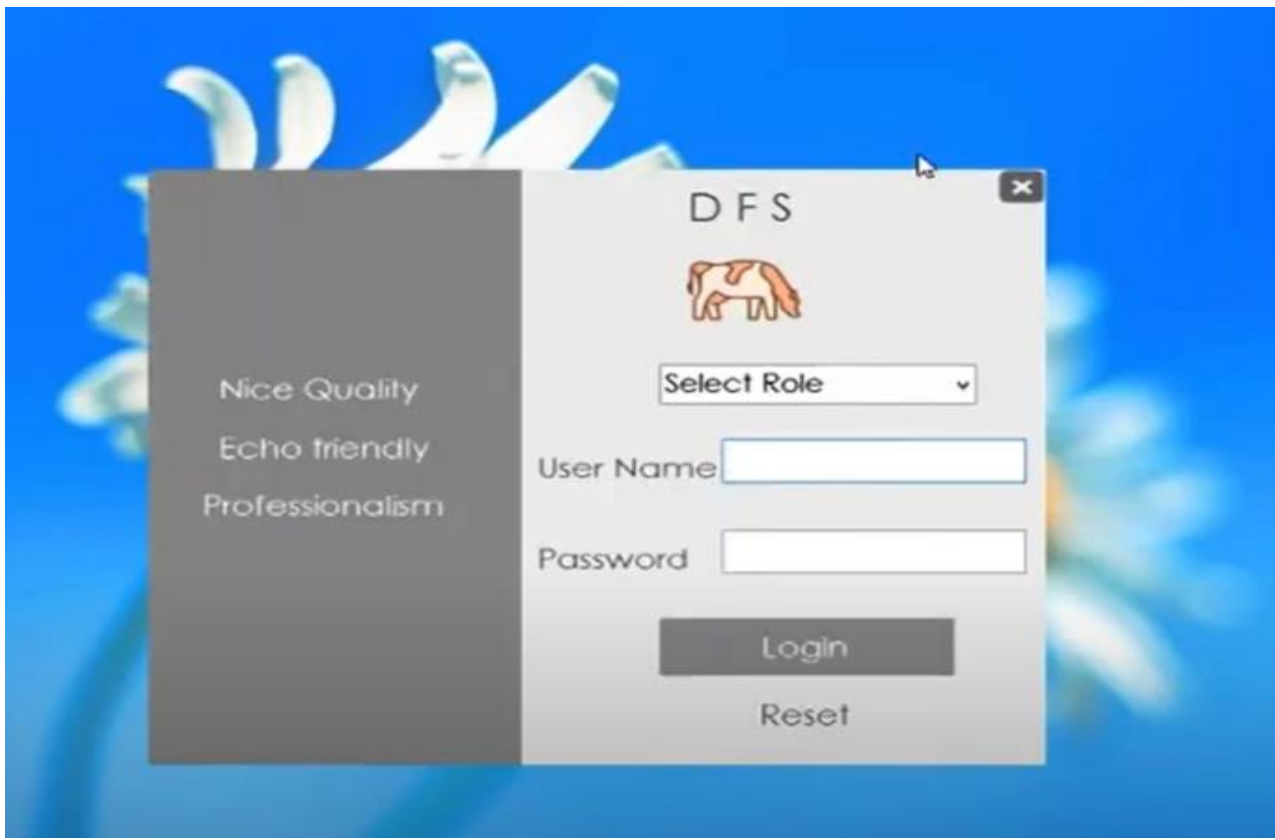
ATTRIBUTES	DATA TYPE	CONSTRAINTS
SId	int	Primary Key
Date	date	Not Null
Uprice	int	Not Null
ClientName	varchar(50)	Not Null
ClientPhone	varchar(50)	Not Null
EmpId	int	Not Null
Quantity	int	Not Null
Amount	int	Not Null

ATTRIBUTES	DATA TYPE	CONSTRAINTS
MId	int	Primary Key
CowId	int	Not Null
CowName	varchar(50)	Not Null
AmMilk	int	Not Null
NoonMilk	int	Not Null
PmMilk	int	Not Null
TotalMilk	int	Not Null
DateProd	date	Not Null

4.3 USER INTERFACE DESIGN



LOGIN PAGE



ADMIN LOGIN

 DFSM

DFSMEmployees

Name

DOB

Gender

Phone

Address

Password

2021-01-22

Save

Edit

Delete

Clear

Employees List

EmpId	EmpName	EmpDob	Gender	Phone	Address	EmpPass
300	Suresh	1995-06-14	Male	3699511584	Bangalore	1234

Employees Login

 DFSM

Cows

Milk Production

Health

Breeding

Milk Sales

Finance

Dashboard

DFSManage Cows

Cow Name

Ear Tag

Color

Breed

Date Of Birth

Age

Weight at Birth

Pasture

2021-01-22

0

Save

Edit

Delete

Clear

Filter Cows List

CowId	CowName	EarTag	Color	Breed	Age	WeightAtBir	Pasture
1000	Cow1	C1	Blue	Friesian	4	120	BA

MANAGE COWS

 DFSM

 Cows

 Milk Production

 Health

 Breeding

 Milk Sales

 Finance

 Dashboard

DFSM

Manage Cows

Cow Name

Queen C4

Ear Tag

Queen C3 I

Color

Red

Breed

Balancer

Date Of Birth

2013-07-11

Age

7

Weight at Birth

110

Pasture

BA

Save

Edit

Delete

Clear

Filter Cows List

CowId	CowName	EarTag	Color	Breed	Age	WeightAtBirth	Pasture
1000	Cow1	C1	Blue	Friesian	4	120	BA
1002	Queen C1	Queen C1	White	Begayt	2	88	Alfalfa
1003	Queen C2	Queen C2	Black	Barzona	2	73	Red Clover
1004	Queen C3	Queen C3	Red	Balancer	7	110	BA

DAILY MILK PRODUCTION

 DFSM

 Cows

 Milk Production

 Health

 Breeding

 Milk Sales

 Finance

 Dashboard

DFSM

Daily Milk Production

Cow Id

1002

Cow Name

Am Milk

Noon Milk

Pm Milk

Total Milk

Date

2021-01-22

Save

Edit

Delete

Clear

Daily Milk

MId	CowId	CowName	AmMilk	NoonMilk	PmMilk	TotalMilk	DateProd
2	1000	Cow1	50	60	90	200	2021-01-19

HEALTH REPORT

 DFS M

 Cows

 Milk Production

 Health

 Breeding

 Milk Sales

 Finance

 Dashboard

DFS M

Health Report

Cow Id

1002

Cow Name

Event

Diagnosis

Date

2021-01-22

Treatment

Cost of Treatment

Vet Name

Save

Edit

Delete

Clear

Health Report

RepId	cowId	cowname	RepDate	Event	Diagnosis	Treatment	Cost	VetName
501	1000	Cow1	2021-01-21	Fever	BVD	Strinacin	1000	Prassad

COW BREEDING

 DFS M

 Cows

 Milk Production

 Health

 Breeding

 Milk Sales

 Finance

 Dashboard

DFS M

Cow Breeding

Heat Date

2021-01-22

Breed Date

2021-01-22

Cow Id

1002

Cow Name

Pregnancy Date

2021-01-22

Expected Date to Calve

2021-01-22

Date Calved

2021-01-22

Cow Age

Remarks

Save

Edit

Delete

Reset

Breed Reports

Brid	HeatDate	BreedDate	CowId	CowName	PregDate	ExpDate	DateCalv	CowAge	Remarks
2001	2021-01...	2021-01...	1000	Cow1	2021-01...	2021-01...	2021-01...	4	Success....

MILK SALES

 DFS M
 Cows
 Milk Production
 Health
 Breeding
Milk Sales
 Finance
 Dashboard

DFS M
Milk Sales

Employee Id

Date

Price

Client Name

Client Phone

Quantity

Total

Sales Report

Sid	Date	Uprice	ClientName	ClientPhone	EmpId	Quantity	Amount
1	2021-01-21	20	Singh	6336544582	300	35	700
2	2021-01-21	20	Bannu	6544568521	300	85	1700
3	2021-01-22	20	Tarun	1233217448	302	30	600
4	2021-01-22	20	Annusha	9633699559	302	35	700

FARM FINANCES

 DFS M
 Cows
 Milk Production
 Health
 Breeding
 Milk Sales
Finance
 Dashboard

DFS M
Farm Finances

Employee

Expenditures
 Date

 Purpose

 Amount

Expenditure List
 Filter

ExpId	ExpDate	ExpPurpose	ExpAmount	EmpId
2	2021-01-21	Tax	3800	1
3	2021-01-20	Maintena...	1280	1
4	2021-01-21	Salary	5000	300

Incomes
 Income Date

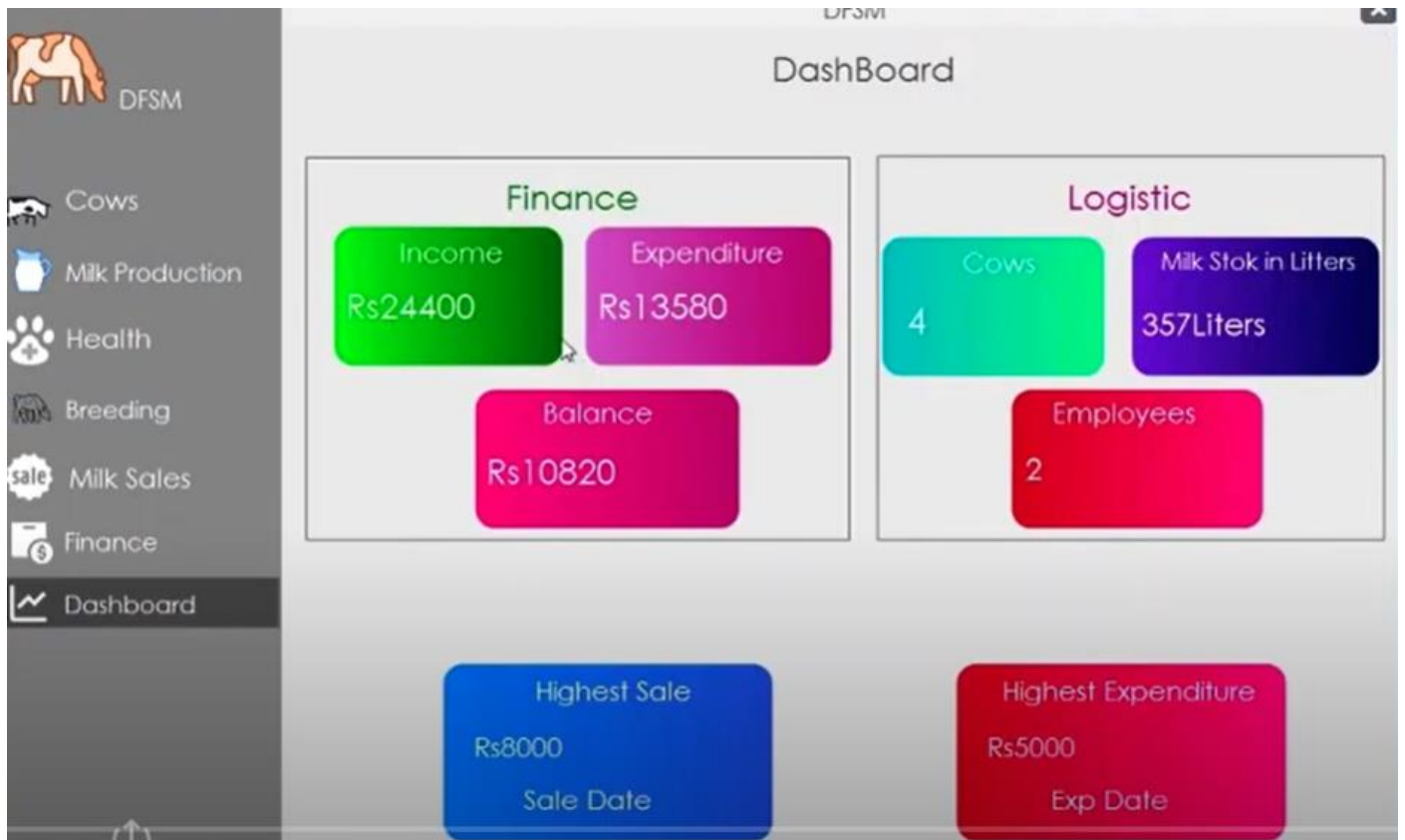
 Type

 Amount

Income List
 Filter

IncId	IncDate	IncPurpose	IncAmt	EmpId
1	2021-01-21	Services	3500	1
2	2021-01-20	Training	8000	1
3	2021-01-21	Sales	1700	300
4	2021-01-21	Services	8000	300
5	2021-01-22	Sales	600	302
6	2021-01-22	Sales	700	302

DASHBOARD



4.4 TEST CASES

SR NO .	Form name	Test Condition	Step or Procedure	Input Test Data	Expected Result	Actual Output	Pass/Fail
1.	Employee Login	Check Login with valid input	Username With wrong password	Username: Shreekesh Password; shreekesh	Display Message “invalid Username Or Password ”	Display Message “invalid Username Or Password ”	Pass
2.	Admin Login	Check Login With valid input	Wrong Username With Correct Password	Username: Arjun Password: Shreekesh 1	Display Message “invalid Username Or Password ”	Display Message “invalid Username Or Password ”	Pass

Chapter 5

5.1 CODING DETAILS AND EFFICIENCY

Breeding.cs:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class Breedings : Form
    {
        public Breedings()
        {
            InitializeComponent();
            populate();
            FillCowId();
        }

        private void panel2_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel3_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel5_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel7_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel6_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel8_Paint(object sender, PaintEventArgs e)
        {
        }

        private void label17_Click(object sender, EventArgs e)
        {
            DashBoard Ob = new DashBoard();
            Ob.Show();
            this.Hide();
        }

        private void label15_Click(object sender, EventArgs e)
        {
        }
    }
}
```

```

        Finances Ob = new Finances();
        Ob.Show();
        this.Hide();
    }

    private void label16_Click(object sender, EventArgs e)
    {
        MilkSales Ob = new MilkSales();
        Ob.Show();
        this.Hide();
    }

    private void label14_Click(object sender, EventArgs e)
    {
        CowHealth Ob = new CowHealth();
        Ob.Show();
        this.Hide();
    }

    private void label6_Click(object sender, EventArgs e)
    {
        MilkProduction Ob = new MilkProduction();
        Ob.Show();
        this.Hide();
    }

    private void label5_Click(object sender, EventArgs e)
    {
        Cows Ob = new Cows();
        Ob.Show();
        this.Hide();
    }
}

SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");
private void FillCowId()
{
    Con.Open();
    SqlCommand cmd = new SqlCommand("select CowId from CowTbl", Con);
    SqlDataReader Rdr;
    Rdr = cmd.ExecuteReader();
    DataTable dt = new DataTable();
    dt.Columns.Add("CowId", typeof(int));
    dt.Load(Rdr);
    CowIdCb.ValueMember = "CowId";
    CowIdCb.DataSource = dt;
    Con.Close();
}

private void populate()
{
    Con.Open();
    string query = "select * from BreedTbl";
    SqlDataAdapter sda = new SqlDataAdapter(query, Con);
    SqlCommandBuilder builder = new SqlCommandBuilder(sda);
    var ds = new DataSet();
    sda.Fill(ds);
    BreedDGV.DataSource = ds.Tables[0];
    Con.Close();
}

private void GetCowName()
{
    Con.Open();
    string query = "select * from CowTbl where CowId=" + CowIdCb.SelectedValue.ToString() +
"";

    SqlCommand cmd = new SqlCommand(query, Con);
    DataTable dt = new DataTable();
    SqlDataAdapter sda = new SqlDataAdapter(cmd);
    sda.Fill(dt);

```

```

        foreach (DataRow dr in dt.Rows)
        {
            CowNameTb.Text = dr["CowName"].ToString();
            CowAgeTb.Text = dr["Age"].ToString();

        }
        Con.Close();
    }
    private void Breedings_Load(object sender, EventArgs e)
    {

    }

    private void CowIdCb_SelectionChangeCommitted(object sender, EventArgs e)
    {
        GetCowName();
    }
    private void Clear()
    {
        CowNameTb.Text = "";
        RemarksTb.Text = "";
        CowAgeTb.Text = "";
        key = 0;
    }
    private void button1_Click(object sender, EventArgs e)
    {
        if (CowIdCb.SelectedIndex == -1 || CowNameTb.Text == "" || RemarksTb.Text == "" ||
CowAgeTb.Text == "")
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "insert into BreedTbl
values('"+HeatDate.Value.Date+"', '"+BreedDate.Value.Date+"', " + CowIdCb.SelectedValue.ToString() +
", '" + CowNameTb.Text + "', '" + PregDate.Value.Date + "', '" + ExpDate.Value.Date + "', '" +
DateCalved.Value.Date + "', " + CowAgeTb.Text + ", '" + RemarksTb.Text + "')";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Breeding Report Saved");

                Con.Close();
                populate();
                Clear();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

    private void button4_Click(object sender, EventArgs e)
    {
        Clear();
    }
    int key = 0;
    private void BreedDGV_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        HeatDate.Text = BreedDGV.SelectedRows[0].Cells[1].Value.ToString();
        BreedDate.Text = BreedDGV.SelectedRows[0].Cells[2].Value.ToString();
        CowIdCb.SelectedValue = BreedDGV.SelectedRows[0].Cells[3].Value.ToString();
        CowNameTb.Text = BreedDGV.SelectedRows[0].Cells[4].Value.ToString();
        PregDate.Text = BreedDGV.SelectedRows[0].Cells[5].Value.ToString();
        ExpDate.Text = BreedDGV.SelectedRows[0].Cells[6].Value.ToString();
        DateCalved.Text = BreedDGV.SelectedRows[0].Cells[7].Value.ToString();
    }

```

```

        CowAgeTb.Text = BreedDGV.SelectedRows[0].Cells[8].Value.ToString();
        RemarksTb.Text = BreedDGV.SelectedRows[0].Cells[9].Value.ToString();
        if (CowNameTb.Text == "")
        {
            key = 0;
        }
        else
        {
            key = Convert.ToInt32(BreedDGV.SelectedRows[0].Cells[0].Value.ToString());
        }
    }

    private void button3_Click(object sender, EventArgs e)
    {
        if (key == 0)
        {
            MessageBox.Show("Select The Breed Report To Be Deleted");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "delete from BreedTbl where BrId=" + key + ";";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Breed Deleted Successfully");
                Con.Close();
                populate();
                Clear();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

    private void button2_Click(object sender, EventArgs e)
    {
        if (CowIdCb.SelectedIndex == -1 || CowNameTb.Text == "" || RemarksTb.Text == "" ||
        CowAgeTb.Text == "")
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "update BreedTbl set HeatDate='" + HeatDate.Value.Date +
                "',BreedDate='" + BreedDate.Value.Date + "',CowId=" + CowIdCb.SelectedValue.ToString() +
                "',CowName='" + CowNameTb.Text + "',PregDate='" + PregDate.Value.Date + "',ExpDateCalve='" +
                ExpDate.Value.Date + "',DateCalved='" + DateCalved.Value.Date + "',CowAge=" + CowAgeTb.Text +
                "',Remarks='" + RemarksTb.Text + "' where BrId=" + key + ";";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Breeding Updated Successfully");
                Con.Close();
                populate();
                Clear();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

```

```

    }

    private void pictureBox9_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }

    private void pictureBox10_Click(object sender, EventArgs e)
    {
        Login Ob = new Login();
        Ob.Show();
        this.Hide();
    }
}
}

```

CowsHealth.cs:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class CowHealth : Form
    {
        public CowHealth()
        {
            InitializeComponent();
            FillCowId();
            populate();
        }

        private void panel2_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel3_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel4_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel7_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel6_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel8_Paint(object sender, PaintEventArgs e)
        {
        }
    }
}

```

```

    }

    private void label17_Click(object sender, EventArgs e)
    {
        DashBoard Ob = new DashBoard();
        Ob.Show();
        this.Hide();
    }

    private void label15_Click(object sender, EventArgs e)
    {
        Finances Ob = new Finances();
        Ob.Show();
        this.Hide();
    }

    private void label16_Click(object sender, EventArgs e)
    {
        MilkSales Ob = new MilkSales();
        Ob.Show();
        this.Hide();
    }

    private void label7_Click(object sender, EventArgs e)
    {
        Breedings Ob = new Breedings();
        Ob.Show();
        this.Hide();
    }

    private void label6_Click(object sender, EventArgs e)
    {
        MilkProduction Ob = new MilkProduction();
        Ob.Show();
        this.Hide();
    }

    private void label5_Click(object sender, EventArgs e)
    {
        Cows Ob = new Cows();
        Ob.Show();
        this.Hide();
    }
}
SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");
private void FillCowId()
{
    Con.Open();
    SqlCommand cmd = new SqlCommand("select CowId from CowTbl", Con);
    SqlDataReader Rdr;
    Rdr = cmd.ExecuteReader();
    DataTable dt = new DataTable();
    dt.Columns.Add("CowId", typeof(int));
    dt.Load(Rdr);
    CowIdCb.ValueMember = "CowId";
    CowIdCb.DataSource = dt;
    Con.Close();
}
private void populate()
{
    Con.Open();
    string query = "select * from HealthTbl";
    SqlDataAdapter sda = new SqlDataAdapter(query, Con);
    SqlCommandBuilder builder = new SqlCommandBuilder(sda);
    var ds = new DataSet();
    sda.Fill(ds);
    HealthDGV.DataSource = ds.Tables[0];
}

```

```

        Con.Close();
    }
    private void GetCowName()
    {
        Con.Open();
        string query = "select * from CowTbl where CowId=" + CowIdCb.SelectedValue.ToString() +
        "";

        SqlCommand cmd = new SqlCommand(query, Con);
        DataTable dt = new DataTable();
        SqlDataAdapter sda = new SqlDataAdapter(cmd);
        sda.Fill(dt);
        foreach (DataRow dr in dt.Rows)
        {
            CowNameTb.Text = dr["CowName"].ToString();
        }
        Con.Close();
    }
    private void CowHealth_Load(object sender, EventArgs e)
    {
    }

    private void CowIdCb_SelectionChangeCommitted(object sender, EventArgs e)
    {
        GetCowName();
    }
    private void Clear()
    {
        CowNameTb.Text = "";
        EventTb.Text = "";
        CostTb.Text = "";
        DiagnosisTb.Text = "";
        VetNameTb.Text = "";
        TreatmentTb.Text = "";
        key = 0;
    }
    private void button1_Click(object sender, EventArgs e)
    {
        if (CowIdCb.SelectedIndex == -1 || CowNameTb.Text == "" || EventTb.Text == "" ||
        CostTb.Text == "" || VetNameTb.Text == "" || DiagnosisTb.Text == "" || TreatmentTb.Text == "")
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "insert into HealthTbl values(" +
                CowIdCb.SelectedValue.ToString() + "," + CowNameTb.Text + "," + Date.Value.Date + "," + EventTb.Text + "," +
                DiagnosisTb.Text + "," + TreatmentTb.Text + "," + CostTb.Text + "," + VetNameTb.Text + ")";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Health Issue Saved");

                Con.Close();
                populate();
                Clear();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

    private void button4_Click(object sender, EventArgs e)

```

```

{
    Clear();
}
int key = 0;
private void HealthDGV_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    CowIdCb.SelectedValue = HealthDGV.SelectedRows[0].Cells[1].Value.ToString();
    CowNameTb.Text = HealthDGV.SelectedRows[0].Cells[2].Value.ToString();
    Date.Text = HealthDGV.SelectedRows[0].Cells[3].Value.ToString();
    EventTb.Text = HealthDGV.SelectedRows[0].Cells[4].Value.ToString();
    DiagnosisTb.Text = HealthDGV.SelectedRows[0].Cells[5].Value.ToString();
    TreatmentTb.Text = HealthDGV.SelectedRows[0].Cells[6].Value.ToString();
    CostTb.Text = HealthDGV.SelectedRows[0].Cells[7].Value.ToString();
    VetNameTb.Text = HealthDGV.SelectedRows[0].Cells[8].Value.ToString();
    if (CowNameTb.Text == "")
    {
        key = 0;
    }
    else
    {
        key = Convert.ToInt32(HealthDGV.SelectedRows[0].Cells[0].Value.ToString());
    }
}

private void button3_Click(object sender, EventArgs e)
{
    if (key == 0)
    {
        MessageBox.Show("Select The Report To Be Deleted");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "delete from HealthTbl where RepId=" + key + ";";
            SqlCommand cmd = new SqlCommand(Query, Con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Report Deleted Successfully");

            Con.Close();
            populate();
            Clear();
        }
        catch (Exception Ex)
        {
            MessageBox.Show(Ex.Message);
        }
    }
}

private void button2_Click(object sender, EventArgs e)
{
    if (CowIdCb.SelectedIndex == -1 || CowNameTb.Text == "" || EventTb.Text == "" ||
    CostTb.Text == "" || VetNameTb.Text == "" || DiagnosisTb.Text == "" || TreatmentTb.Text == "")
    {
        MessageBox.Show("Missing Information");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "update HealthTbl set CowId=" + CowIdCb.SelectedValue.ToString()
+ ",cowname='" + CowNameTb.Text + "',RepDate='" + Date.Value.Date + "',Event='" + EventTb.Text +

```

```

        "',Diagnosis='" + DiagnosisTb.Text + "',Treatment='" + TreatmentTb.Text + "',Cost='" + CostTb.Text +
        "',VetName='" + VetNameTb.Text + "'" where RepId=" + key + "''";
        SqlCommand cmd = new SqlCommand(Query, Con);
        cmd.ExecuteNonQuery();
        MessageBox.Show("Product Updated Successfully");
        Con.Close();
        populate();
        Clear();
    }
    catch (Exception Ex)
    {
        MessageBox.Show(Ex.Message);
    }
}

private void pictureBox9_Click(object sender, EventArgs e)
{
    Application.Exit();
}

private void pictureBox10_Click(object sender, EventArgs e)
{
    Login Ob = new Login();
    Ob.Show();
    this.Hide();
}
}
}

```

Cows:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Data.SqlClient;

namespace DairyFarmSystem
{
    public partial class Cows : Form
    {
        public Cows()
        {
            InitializeComponent();
            populate();
        }

        SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");

        private void panel3_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel5_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel4_Paint(object sender, PaintEventArgs e)
        {
        }
    }
}

```

```

    }

    private void panel7_Paint(object sender, PaintEventArgs e)
    {

    }

    private void panel6_Paint(object sender, PaintEventArgs e)
    {

    }

    private void panel8_Paint(object sender, PaintEventArgs e)
    {

    }

    private void label17_Click(object sender, EventArgs e)
    {
        DashBoard Ob = new DashBoard();
        Ob.Show();
        this.Hide();
    }

    private void label15_Click(object sender, EventArgs e)
    {
        Finances Ob = new Finances();
        Ob.Show();
        this.Hide();
    }

    private void label16_Click(object sender, EventArgs e)
    {
        MilkSales Ob = new MilkSales();
        Ob.Show();
        this.Hide();
    }

    private void label7_Click(object sender, EventArgs e)
    {
        Breedings Ob = new Breedings();
        Ob.Show();
        this.Hide();
    }

    private void label14_Click(object sender, EventArgs e)
    {
        CowHealth Ob = new CowHealth();
        Ob.Show();
        this.Hide();
    }

    private void label6_Click(object sender, EventArgs e)
    {
        MilkProduction Ob = new MilkProduction();
        Ob.Show();
        this.Hide();
    }
}
private void populate()
{
    Con.Open();
    string query = "select * from Cowtbl";
    SqlDataAdapter sda = new SqlDataAdapter(query, Con);
    SqlCommandBuilder builder = new SqlCommandBuilder(sda);
    var ds = new DataSet();
    sda.Fill(ds);
    CowsDGV.DataSource = ds.Tables[0];
    Con.Close();
}

```

```

    }
    private void Clear()
    {
        CowNameTb.Text = "";
        EarTagTb.Text = "";
        ColorTb.Text = "";
        BreedTb.Text = "";
        WeigthTb.Text = "";
        AgeTb.Text = "";
        PastureTb.Text = "";
        key = 0;
    }
    int age = 0;
    private void button1_Click(object sender, EventArgs e)
    {
        if (CowNameTb.Text==" " || EarTagTb.Text == " "||ColorTb.Text == " "||BreedTb.Text ==
""||WeigthTb.Text =="" || AgeTb.Text == " " || PastureTb.Text == "")
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "insert into CowTbl values('" + CowNameTb.Text + "',' +
EarTagTb.Text + "',' + ColorTb.Text + "',' + BreedTb.Text + "',' + age + "','" + WeigthTb.Text +
',' + PastureTb.Text + "')";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Cow Saved Successfully");

                Con.Close();
                populate();
                Clear();
            }catch(Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

    private void Cows_Load(object sender, EventArgs e)
    {

    }

    private void DOBDate_ValueChanged(object sender, EventArgs e)
    {
        age = Convert.ToInt32((DateTime.Today.Date-DOBDate.Value.Date).Days)/365;
    }

    private void DOBDate_MouseLeave(object sender, EventArgs e)
    {
        age = Convert.ToInt32((DateTime.Today.Date - DOBDate.Value.Date).Days) / 365;
        AgeTb.Text = "" + age;
    }

    private void button4_Click(object sender, EventArgs e)
    {
        Clear();
    }
    int key = 0;
    private void CowsDGV_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        CowNameTb.Text = CowsDGV.SelectedRows[0].Cells[1].Value.ToString();
        EarTagTb.Text = CowsDGV.SelectedRows[0].Cells[2].Value.ToString();
    }

```

```

        ColorTb.Text = CowsDGV.SelectedRows[0].Cells[3].Value.ToString();
        BreedTb.Text = CowsDGV.SelectedRows[0].Cells[4].Value.ToString();
        WeighTb.Text = CowsDGV.SelectedRows[0].Cells[6].Value.ToString();
        PastureTb.Text = CowsDGV.SelectedRows[0].Cells[7].Value.ToString();
        if(CowNameTb.Text == "")
        {
            key = 0;
            age = 0;
        }else
        {
            key = Convert.ToInt32(CowsDGV.SelectedRows[0].Cells[0].Value.ToString());
            age = Convert.ToInt32(CowsDGV.SelectedRows[0].Cells[5].Value.ToString());
        }
    }

private void button3_Click(object sender, EventArgs e)
{
    if (key == 0)
    {
        MessageBox.Show("Select The Cow To Be Deleted");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "delete from CowTbl where CowId=" + key + ";";
            SqlCommand cmd = new SqlCommand(Query, Con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Cow Deleted Successfully");

            Con.Close();
            populate();
            Clear();
        }
        catch (Exception Ex)
        {
            MessageBox.Show(Ex.Message);
        }
    }
}

private void button2_Click(object sender, EventArgs e)
{
    if (CowNameTb.Text == "" || EarTagTb.Text == "" || ColorTb.Text == "" || BreedTb.Text == "" || WeighTb.Text == "" || AgeTb.Text == "" || PastureTb.Text == "")
    {
        MessageBox.Show("Missing Information");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "update CowTbl set CowName='" + CowNameTb.Text + "',EarTag='" + EarTagTb.Text + "',Color='" + ColorTb.Text + "',Breed='" + BreedTb.Text + "',Age=" + age + ",weightatbirth=" + WeighTb.Text + ",Pasture='" + PastureTb.Text + "' where Cowid=" + key + ";";
            SqlCommand cmd = new SqlCommand(Query, Con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Cow Updated Successfully");
            Con.Close();
            populate();
            Clear();
        }
        catch (Exception Ex)
        {
            MessageBox.Show(Ex.Message);
        }
    }
}

```

```

    }
}

private void pictureBox9_Click(object sender, EventArgs e)
{
    Application.Exit();
}
private void SearchCow()
{
    Con.Open();
    string query = "select * from Cowtbl where CowName like '%" + CowSearchTb.Text + "%'";
    SqlDataAdapter sda = new SqlDataAdapter(query, Con);
    SqlCommandBuilder builder = new SqlCommandBuilder(sda);
    var ds = new DataSet();
    sda.Fill(ds);
    CowsDGV.DataSource = ds.Tables[0];
    Con.Close();
}

private void CowSearchTb_OnValueChanged(object sender, EventArgs e)
{
    SearchCow();
}

private void pictureBox10_Click(object sender, EventArgs e)
{
    Login Ob = new Login();
    Ob.Show();
    this.Hide();
}
}
}

```

Dashboard:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class DashBoard : Form
    {
        public DashBoard()
        {
            InitializeComponent();
            Finance();
            Logistic();
            GetMax();
        }

        private void panel2_Paint(object sender, PaintEventArgs e)
        {

        }

        private void panel3_Paint(object sender, PaintEventArgs e)
        {

```

```

    }

    private void panel5_Paint(object sender, PaintEventArgs e)
    {

    }

    private void panel4_Paint(object sender, PaintEventArgs e)
    {

    }

    private void panel7_Paint(object sender, PaintEventArgs e)
    {

    }

    private void panel6_Paint(object sender, PaintEventArgs e)
    {

    }

    private void label15_Click(object sender, EventArgs e)
    {
        Finances Ob = new Finances();
        Ob.Show();
        this.Hide();
    }

    private void label16_Click(object sender, EventArgs e)
    {

        MilkSales Ob = new MilkSales();
        Ob.Show();
        this.Hide();
    }

    private void label7_Click(object sender, EventArgs e)
    {
        Breedings Ob = new Breedings();
        Ob.Show();
        this.Hide();
    }

    private void label14_Click(object sender, EventArgs e)
    {
        CowHealth Ob = new CowHealth();
        Ob.Show();
        this.Hide();
    }

    private void label6_Click(object sender, EventArgs e)
    {
        MilkProduction Ob = new MilkProduction();
        Ob.Show();
        this.Hide();
    }

    private void label5_Click(object sender, EventArgs e)
    {
        Cows Ob = new Cows();
        Ob.Show();
        this.Hide();
    }

    private void panel8_Paint(object sender, PaintEventArgs e)
    {

    }

```

```

private void label17_Click(object sender, EventArgs e)
{
}

SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");
private void Finance()
{
    //We calculate the Finance related Analytics
    Con.Open();
    SqlDataAdapter sda = new SqlDataAdapter("select sum(IncAmt) from Incometbl", Con);
    SqlDataAdapter sda1 = new SqlDataAdapter("select sum(ExpAmount) from ExpenditureTbl",
Con);

    DataTable dt = new DataTable();
    sda.Fill(dt);
    int inc, exp;
    double bal;
    //inc = Convert.ToInt32(dt.Rows[0][0].ToString());
    if (Int32.TryParse(dt.Rows[0][0].ToString(), out inc))
    {
        //Conversion successfull
    }
    else
    {
        //handle the error
    }

    IncLbl.Text = "Rs"+dt.Rows[0][0].ToString();
    DataTable dt1 = new DataTable();
    sda1.Fill(dt1);
    //exp = Convert.ToInt32(dt1.Rows[0][0].ToString());
    if (Int32.TryParse(dt1.Rows[0][0].ToString(), out exp))
    {
        //Conversion successfull
    }
    else
    {
        //handle the error
    }

    bal = inc - exp;
    ExpLbl.Text = "Rs"+dt1.Rows[0][0].ToString();
    BalLbl.Text = "Rs" + bal;
    Con.Close();
}

private void Logistic()
{
    //We calculate the Logistics related Analytics
    Con.Open();
    SqlDataAdapter sda = new SqlDataAdapter("select Count(*) from Cowtbl", Con);
    SqlDataAdapter sda1 = new SqlDataAdapter("select sum(TotalMilk) from MilkTbl", Con);
    SqlDataAdapter sda2 = new SqlDataAdapter("select Count(*) from EmployeeTbl", Con);
    DataTable dt = new DataTable();
    sda.Fill(dt);
    Cownumlbl.Text = dt.Rows[0][0].ToString();
    DataTable dt1 = new DataTable();
    sda1.Fill(dt1);
    MilkLbl.Text = dt1.Rows[0][0].ToString()+ "Liters";
    DataTable dt2 = new DataTable();
    sda2.Fill(dt2);
    Empnumlbl.Text = dt2.Rows[0][0].ToString();
    Con.Close();
}

private void GetMax()
{

```

```

        SqlDataAdapter sda = new SqlDataAdapter("select Max(IncAmt) from Incometbl ", Con);
        SqlDataAdapter sda1 = new SqlDataAdapter("select Max(ExpAmount) from ExpenditureTbl",
Con);

        DataTable dt = new DataTable();
        sda.Fill(dt);
        DataTable dt1 = new DataTable();
        sda1.Fill(dt1);
        HighAmtLbl.Text = "Rs" + dt.Rows[0][0].ToString();
        //HighDateLbl.Text =dt.Rows[0][1].ToString();
        HighExpLbl.Text = "Rs" + dt1.Rows[0][0].ToString();

    }
    private void DashBoard_Load(object sender, EventArgs e)
    {

    }

    private void pictureBox9_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }

    private void pictureBox10_Click(object sender, EventArgs e)
    {
        Login Ob = new Login();
        Ob.Show();
        this.Hide();
    }

    private void guna2GradientPanel9_Paint(object sender, PaintEventArgs e)
    {

    }
}
}

```

Employees.cs:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class Employees : Form
    {
        public Employees()
        {
            InitializeComponent();
            populate();
        }

        SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");
        private void populate()
        {
            Con.Open();
            string query = "select * from EmployeeTbl";
            SqlDataAdapter sda = new SqlDataAdapter(query, Con);
            SqlCommandBuilder builder = new SqlCommandBuilder(sda);

```

```

        var ds = new DataSet();
        sda.Fill(ds);
        EmployeeDGV.DataSource = ds.Tables[0];
        Con.Close();
    }
    private void Clear()
    {
        PhoneTb.Text = "";
        EmpNameTb.Text = "";
        AddressTb.Text = "";
        GenCb.SelectedIndex = -1;
        key = 0;
        EmpPassTb.Text = "";
    }
    private void button1_Click(object sender, EventArgs e)
    {
        if (EmpNameTb.Text == "" || GenCb.SelectedIndex == -1 || PhoneTb.Text == "" ||
        AddressTb.Text == "" || EmpPassTb.Text == "")
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "insert into EmployeeTbl values('" + EmpNameTb.Text + "','" +
                DOB.Value.Date + "','" + GenCb.SelectedItem.ToString() + "','" + PhoneTb.Text + "','" +
                AddressTb.Text + "','" + EmpPassTb.Text + "')";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Employee Saved Successfully");

                Con.Close();
                populate();
                Clear();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }
    int key = 0;
    private void EmployeeDGV_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        EmpNameTb.Text = EmployeeDGV.SelectedRows[0].Cells[1].Value.ToString();
        DOB.Text = EmployeeDGV.SelectedRows[0].Cells[2].Value.ToString();
        GenCb.SelectedItem = EmployeeDGV.SelectedRows[0].Cells[3].Value.ToString();
        PhoneTb.Text = EmployeeDGV.SelectedRows[0].Cells[4].Value.ToString();
        AddressTb.Text = EmployeeDGV.SelectedRows[0].Cells[5].Value.ToString();
        EmpPassTb.Text = EmployeeDGV.SelectedRows[0].Cells[6].Value.ToString();
        if (EmpNameTb.Text == "")
        {
            key = 0;
        }
        else
        {
            key = Convert.ToInt32(EmployeeDGV.SelectedRows[0].Cells[0].Value.ToString());
        }
    }
    private void button4_Click(object sender, EventArgs e)
    {
        Clear();
    }

```

```

private void button3_Click(object sender, EventArgs e)
{
    if (key == 0)
    {
        MessageBox.Show("Select The Employee To Be Deleted");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "delete from EmployeeTbl where EmpId=" + key + ";";
            SqlCommand cmd = new SqlCommand(Query, Con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Employee Deleted Successfully");

            Con.Close();
            populate();
            Clear();
        }
        catch (Exception Ex)
        {
            MessageBox.Show(Ex.Message);
        }
    }
}

private void button2_Click(object sender, EventArgs e)
{
    if (EmpNameTb.Text == "" || GenCb.SelectedIndex == -1 || PhoneTb.Text == "" ||
AddressTb.Text == "")
    {
        MessageBox.Show("Missing Information");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "update EmployeeTbl set EmpName='" + EmpNameTb.Text +
            "',EmpDOB='" + DOB.Value.Date + "',Gender='" + GenCb.SelectedItem.ToString() + "',Phone='" +
            PhoneTb.Text + "',Address='" + AddressTb.Text + "',EmpPass='"+EmpPassTb.Text+"'' where Empid=" + key
            + ";";

            SqlCommand cmd = new SqlCommand(Query, Con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Employee Updated Successfully");
            Con.Close();
            populate();
            Clear();
        }
        catch (Exception Ex)
        {
            MessageBox.Show(Ex.Message);
        }
    }
}

private void pictureBox9_Click(object sender, EventArgs e)
{
    Application.Exit();
}

private void pictureBox10_Click(object sender, EventArgs e)
{
    Login Ob = new Login();
    Ob.Show();
    this.Hide();
}

```

```
}  
}
```

Finances.cs:

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Data.SqlClient;  
using System.Drawing;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows.Forms;  
  
namespace DairyFarmSystem  
{  
    public partial class Finances : Form  
    {  
        public Finances()  
        {  
            InitializeComponent();  
            populateExp();  
            populateInc();  
            FillEmpId();  
        }  
  
        private void panel8_Paint(object sender, PaintEventArgs e)  
        {  
  
        }  
  
        private void panel7_Paint(object sender, PaintEventArgs e)  
        {  
  
        }  
  
        private void panel4_Paint(object sender, PaintEventArgs e)  
        {  
  
        }  
  
        private void panel5_Paint(object sender, PaintEventArgs e)  
        {  
  
        }  
  
        private void panel3_Paint(object sender, PaintEventArgs e)  
        {  
  
        }  
  
        private void panel2_Paint(object sender, PaintEventArgs e)  
        {  
  
        }  
  
        private void label17_Click(object sender, EventArgs e)  
        {  
            DashBoard Ob = new DashBoard();  
            Ob.Show();  
            this.Hide();  
        }  
  
        private void label16_Click(object sender, EventArgs e)  
        {  

```

```

        MilkSales Ob = new MilkSales();
        Ob.Show();
        this.Hide();
    }

    private void label7_Click(object sender, EventArgs e)
    {
        Breedings Ob = new Breedings();
        Ob.Show();
        this.Hide();
    }

    private void label14_Click(object sender, EventArgs e)
    {
        CowHealth Ob = new CowHealth();
        Ob.Show();
        this.Hide();
    }

    private void label6_Click(object sender, EventArgs e)
    {
        MilkProduction Ob = new MilkProduction();
        Ob.Show();
        this.Hide();
    }

    private void label5_Click(object sender, EventArgs e)
    {
        DashBoard Ob = new DashBoard();
        Ob.Show();
        this.Hide();
    }
}
SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");

    private void populateExp()
    {
        Con.Open();
        string query = "select * from ExpenditureTbl";
        SqlDataAdapter sda = new SqlDataAdapter(query, Con);
        SqlCommandBuilder builder = new SqlCommandBuilder(sda);
        var ds = new DataSet();
        sda.Fill(ds);
        ExpDGV.DataSource = ds.Tables[0];
        Con.Close();
    }

    private void populateInc()
    {
        Con.Open();
        string query = "select * from IncomeTbl";
        SqlDataAdapter sda = new SqlDataAdapter(query, Con);
        SqlCommandBuilder builder = new SqlCommandBuilder(sda);
        var ds = new DataSet();
        sda.Fill(ds);
        IncDGV.DataSource = ds.Tables[0];
        Con.Close();
    }

    private void FilterIncome()
    {
        Con.Open();
        string query = "select * from IncomeTbl where IncDate='" + IncomeDateFilter.Value.Date +
""";

        SqlDataAdapter sda = new SqlDataAdapter(query, Con);
        SqlCommandBuilder builder = new SqlCommandBuilder(sda);
        var ds = new DataSet();
        sda.Fill(ds);
        IncDGV.DataSource = ds.Tables[0];
        Con.Close();
    }

```

```

    }
    private void FilterExp()
    {
        Con.Open();
        string query = "select * from ExpenditureTbl where ExpDate='" + ExpDateFilter.Value.Date
+ "'";
        SqlDataAdapter sda = new SqlDataAdapter(query, Con);
        SqlCommandBuilder builder = new SqlCommandBuilder(sda);
        var ds = new DataSet();
        sda.Fill(ds);
        ExpDGV.DataSource = ds.Tables[0];
        Con.Close();
    }
    private void ClearExp()
    {
        AmountTb.Text = "";
    }
    private void button5_Click(object sender, EventArgs e)
    {
        if (PurpCb.SelectedIndex == -1 || AmountTb.Text == "" || EmpIdCb.SelectedIndex == -1)
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "insert into ExpenditureTbl values('" + ExpDate.Value.Date +
+ "','" + PurpCb.SelectedItem.ToString() + "','" + AmountTb.Text + "','" +
+ EmpIdCb.SelectedValue.ToString() + "')";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Income Saved Successfully");

                Con.Close();
                populateExp();
                ClearExp();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }
    private void FillEmpId()
    {
        Con.Open();
        SqlCommand cmd = new SqlCommand("select EmpId from EmployeeTbl", Con);
        SqlDataReader Rdr;
        Rdr = cmd.ExecuteReader();
        DataTable dt = new DataTable();
        dt.Columns.Add("EmpId", typeof(int));
        dt.Load(Rdr);
        EmpIdCb.ValueMember = "EmpId";
        EmpIdCb.DataSource = dt;
        Con.Close();
    }
    private void ClearInc()
    {
        IncPurCb.SelectedIndex = -1;
        IncAmount.Text = "";
    }
    private void button1_Click(object sender, EventArgs e)
    {
        if (IncPurCb.SelectedIndex == -1 || IncAmount.Text == "" || EmpIdCb.SelectedIndex == -1)
        {
            MessageBox.Show("Missing Information");
        }
    }

```

```

        else
        {
            try
            {
                Con.Open();
                string Query = "insert into IncomeTbl values('" + IncDate.Value.Date + "','" +
IncPurCb.SelectedItem.ToString() + "','" + IncAmount.Text + "','" + EmpIdCb.SelectedValue.ToString() +
")";

                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Income Saved Successfully");

                Con.Close();
                populateInc();
                ClearInc();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

    private void dateTimePicker3_ValueChanged(object sender, EventArgs e)
    {
        FilterIncome();
    }

    private void pictureBox9_Click(object sender, EventArgs e)
    {
        populateInc();
    }

    private void ExpDateFilter_ValueChanged(object sender, EventArgs e)
    {
        FilterExp();
    }

    private void pictureBox10_Click(object sender, EventArgs e)
    {
        populateExp();
    }

    private void Finances_Load(object sender, EventArgs e)
    {
    }

    private void pictureBox11_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }

    private void pictureBox12_Click(object sender, EventArgs e)
    {
        Login Ob = new Login();
        Ob.Show();
        this.Hide();
    }
}
}

```

Login.cs:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;

```

```

using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class Login : Form
    {
        public Login()
        {
            InitializeComponent();

            private void label4_Click(object sender, EventArgs e)
            {
                UnameTb.Text = "";
                PasswordTb.Text = "";
            }

            SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");
            private void button1_Click(object sender, EventArgs e)
            {
                if (UnameTb.Text == "" || PasswordTb.Text == "")
                {
                    MessageBox.Show("Enter The UserName And Password");
                }
                else
                {
                    if (RoleCb.SelectedIndex > -1)
                    {
                        if (RoleCb.SelectedItem.ToString() == "Admin")
                        {
                            if (UnameTb.Text == "Shreekesh" && PasswordTb.Text == "Shree1")
                            {
                                Employees prod = new Employees();
                                prod.Show();
                                this.Hide();
                            }
                            else
                            {
                                PasswordTb.Text = "";
                                UnameTb.Text = "";
                                PasswordTb.Text = "";
                            }
                        }
                        else
                        {
                            Con.Open();
                            SqlDataAdapter sda = new SqlDataAdapter("Select count(*) from EmployeeTbl
where EmpName='" + UnameTb.Text + "' and EmpPass='" + PasswordTb.Text + "'", Con);
                            DataTable dt = new DataTable();
                            sda.Fill(dt);
                            if (dt.Rows[0][0].ToString() == "1")
                            {
                                Cows cow = new Cows();
                                cow.Show();
                                this.Hide();
                                Con.Close();
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

                else
                {
                    MessageBox.Show("Wrong UserName or Password");
                    UnameTb.Text = "";
                    PasswordTb.Text = "";
                }
                Con.Close();
            }

        }
        else
        {
            MessageBox.Show("Select A Role");
        }
    }

}

private void pictureBox9_Click(object sender, EventArgs e)
{
    Application.Exit();
}
}
}

```

Milk Productions:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class MilkProduction : Form
    {
        public MilkProduction()
        {
            InitializeComponent();
            FillCowId();
            populate();
        }

        private void panel2_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel15_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel14_Paint(object sender, PaintEventArgs e)
        {
        }

        private void panel17_Paint(object sender, PaintEventArgs e)
        {
        }
    }
}

```

```

    }

    private void panel6_Paint(object sender, PaintEventArgs e)
    {

    }

    private void panel8_Paint(object sender, PaintEventArgs e)
    {

    }

    private void label17_Click(object sender, EventArgs e)
    {
        DashBoard Ob = new DashBoard();
        Ob.Show();
        this.Hide();
    }

    private void label15_Click(object sender, EventArgs e)
    {
        Finances Ob = new Finances();
        Ob.Show();
        this.Hide();
    }

    private void label16_Click(object sender, EventArgs e)
    {
        MilkSales Ob = new MilkSales();
        Ob.Show();
        this.Hide();
    }

    private void label7_Click(object sender, EventArgs e)
    {
        Breedings Ob = new Breedings();
        Ob.Show();
        this.Hide();
    }

    private void label14_Click(object sender, EventArgs e)
    {
        CowHealth Ob = new CowHealth();
        Ob.Show();
        this.Hide();
    }

    private void label5_Click(object sender, EventArgs e)
    {
        Cows Ob = new Cows();
        Ob.Show();
        this.Hide();
    }

    private void MilkProduction_Load(object sender, EventArgs e)
    {

    }

    SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");
    private void FillCowId()
    {
        Con.Open();
        SqlCommand cmd = new SqlCommand("select CowId from CowTbl", Con);
        SqlDataReader Rdr;
        Rdr = cmd.ExecuteReader();
        DataTable dt = new DataTable();

```

```

        dt.Columns.Add("CowId", typeof(int));
        dt.Load(Rdr);
        CowIdCb.ValueMember = "CowId";
        CowIdCb.DataSource = dt;
        Con.Close();
    }
    private void populate()
    {
        Con.Open();
        string query = "select * from MilkTbl";
        SqlDataAdapter sda = new SqlDataAdapter(query, Con);
        SqlCommandBuilder builder = new SqlCommandBuilder(sda);
        var ds = new DataSet();
        sda.Fill(ds);
        MilkDGV.DataSource = ds.Tables[0];
        Con.Close();
    }
    private void Clear()
    {
        Cownmametb.Text = "";
        Amtb.Text = "";
        noonTb.Text = "";
        PmTb.Text = "";
        TotalTb.Text = "";
        key = 0;
    }
    private void GetCowName()
    {
        Con.Open();
        string query = "select * from CowTbl where CowId="+CowIdCb.SelectedValue.ToString()+"";
        SqlCommand cmd = new SqlCommand(query, Con);
        DataTable dt = new DataTable();
        SqlDataAdapter sda = new SqlDataAdapter(cmd);
        sda.Fill(dt);
        foreach(DataRow dr in dt.Rows)
        {
            Cownmametb.Text = dr["CowName"].ToString();
        }
        Con.Close();
    }
    private void button1_Click(object sender, EventArgs e)
    {
        if (CowIdCb.SelectedIndex == -1 || Cownmametb.Text == "" || Amtb.Text == "" || PmTb.Text == "" || noonTb.Text == "" || TotalTb.Text == "" )
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "insert into MilkTbl values(" + CowIdCb.SelectedValue.ToString() + "," + Cownmametb.Text + "," + Amtb.Text + "," + noonTb.Text + "," + PmTb.Text + "," + TotalTb.Text + "," + Date.Value.Date + ")";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Milk Saved Successfully");

                Con.Close();
                populate();
                Clear();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

```

```

    }
}

private void button4_Click(object sender, EventArgs e)
{
    Clear();
}

private void CowIdCb_SelectionChangeCommitted(object sender, EventArgs e)
{
    GetCowName();
}

private void PmTb_OnValueChanged(object sender, EventArgs e)
{
}

private void TotalTb_MouseClick(object sender, MouseEventArgs e)
{
}

private void TotalTb_Click(object sender, EventArgs e)
{
}

private void TotalTb_MouseEnter(object sender, EventArgs e)
{
}

private void TotalTb_MouseDown(object sender, MouseEventArgs e)
{
}

private void PmTb_Leave(object sender, EventArgs e)
{
    int total = Convert.ToInt32(Amtb.Text) + Convert.ToInt32(PmTb.Text) +
Convert.ToInt32(noonTb.Text);
    TotalTb.Text = "" + total;
}
int key = 0;
private void MilkDGV_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    CowIdCb.SelectedValue = MilkDGV.SelectedRows[0].Cells[1].Value.ToString();
    Cownmametb.Text = MilkDGV.SelectedRows[0].Cells[2].Value.ToString();
    Amtb.Text = MilkDGV.SelectedRows[0].Cells[3].Value.ToString();
    noonTb.Text = MilkDGV.SelectedRows[0].Cells[4].Value.ToString();
    PmTb.Text = MilkDGV.SelectedRows[0].Cells[5].Value.ToString();
    TotalTb.Text = MilkDGV.SelectedRows[0].Cells[6].Value.ToString();
    Date.Text = MilkDGV.SelectedRows[0].Cells[7].Value.ToString();
    if (Cownmametb.Text == "")
    {
        key = 0;
    }
    else
    {
        key = Convert.ToInt32(MilkDGV.SelectedRows[0].Cells[0].Value.ToString());
    }
}
}

```

```

private void button3_Click(object sender, EventArgs e)
{
    if (key == 0)
    {
        MessageBox.Show("Select The Milk Product To Be Deleted");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "delete from MilkTbl where MId=" + key + ";";
            SqlCommand cmd = new SqlCommand(Query, Con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Product Deleted Successfully");

            Con.Close();
            populate();
            Clear();
        }
        catch (Exception Ex)
        {
            MessageBox.Show(Ex.Message);
        }
    }
}

private void button2_Click(object sender, EventArgs e)
{
    if (CowIdCb.SelectedIndex == -1 || Cownmametb.Text == "" || Amtb.Text == "" || PmTb.Text == "" || noonTb.Text == "" || TotalTb.Text == "")
    {
        MessageBox.Show("Missing Information");
    }
    else
    {
        try
        {
            Con.Open();
            string Query = "update MilkTbl set CowName='" + Cownmametb.Text + "',AmMilk=" + Amtb.Text + ",NoonMilk=" + noonTb.Text + ",PmMilk=" + PmTb.Text + ",TotalMilk=" + TotalTb.Text + ",DateProd='" + Date.Value.Date + "' where Mid=" + key + ";";
            SqlCommand cmd = new SqlCommand(Query, Con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Product Updated Successfully");
            Con.Close();
            populate();
            Clear();
        }
        catch (Exception Ex)
        {
            MessageBox.Show(Ex.Message);
        }
    }
}

private void pictureBox9_Click(object sender, EventArgs e)
{
    Application.Exit();
}

private void pictureBox10_Click(object sender, EventArgs e)
{
    Login Ob = new Login();
    Ob.Show();
    this.Hide();
}
}

```

Milk Sales:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class MilkSales : Form
    {
        public MilkSales()
        {
            InitializeComponent();
            FillEmpId();
            populate();
        }

        private void panel2_Paint(object sender, PaintEventArgs e)
        {

        }

        private void panel3_Paint(object sender, PaintEventArgs e)
        {

        }

        private void panel5_Paint(object sender, PaintEventArgs e)
        {

        }

        private void panel4_Paint(object sender, PaintEventArgs e)
        {

        }

        private void panel6_Paint(object sender, PaintEventArgs e)
        {

        }

        private void panel8_Paint(object sender, PaintEventArgs e)
        {

        }

        private void label17_Click(object sender, EventArgs e)
        {
            DashBoard Ob = new DashBoard();
            Ob.Show();
            this.Hide();
        }

        private void label15_Click(object sender, EventArgs e)
        {
            Finances Ob = new Finances();
            Ob.Show();
            this.Hide();
        }
    }
}
```

```

private void label7_Click(object sender, EventArgs e)
{
    Breedings Ob = new Breedings();
    Ob.Show();
    this.Hide();
}

private void label14_Click(object sender, EventArgs e)
{
    CowHealth Ob = new CowHealth();
    Ob.Show();
    this.Hide();
}

private void label6_Click(object sender, EventArgs e)
{
    MilkProduction Ob = new MilkProduction();
    Ob.Show();
    this.Hide();
}

private void label5_Click(object sender, EventArgs e)
{
    Cows Ob = new Cows();
    Ob.Show();
    this.Hide();
}

SqlConnection Con = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\yhris\Documents\DairyFarmSystem10.mdf;Integr
ated Security=True;Connect Timeout=30");
private void FillEmpId()
{
    Con.Open();
    SqlCommand cmd = new SqlCommand("select EmpId from EmployeeTbl", Con);
    SqlDataReader Rdr;
    Rdr = cmd.ExecuteReader();
    DataTable dt = new DataTable();
    dt.Columns.Add("EmpId", typeof(int));
    dt.Load(Rdr);
    EmpIdCb.ValueMember = "EmpId";
    EmpIdCb.DataSource = dt;
    Con.Close();
}

private void MilkSales_Load(object sender, EventArgs e)
{
}

private void QuantityTb_Leave(object sender, EventArgs e)
{
    int total = Convert.ToInt32(PriceTb.Text) * Convert.ToInt32(QuantityTb.Text);
    TotalTb.Text = "" + total;
}

private void populate()
{
    Con.Open();
    string query = "select * from MilkSalesTbl";
    SqlDataAdapter sda = new SqlDataAdapter(query, Con);
    SqlCommandBuilder builder = new SqlCommandBuilder(sda);
    var ds = new DataSet();
    sda.Fill(ds);
    SalesDGV.DataSource = ds.Tables[0];
    Con.Close();
}

private void Clear()
{
    PriceTb.Text = "";
    QuantityTb.Text = "";
    ClientNameTb.Text = "";
}

```

```

        PhoneTb.Text = "";
        TotalTb.Text = "";
    }
    private void SaveTransaction()
    {
        {
            try
            {
                string Sales = "Sales";
                Con.Open();
                string Query = "insert into IncomeTbl values('" + Date.Value.Date + "',' +
Sales + "',' + TotalTb.Text + "',' + EmpIdCb.SelectedValue.ToString() + ")";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Income Saved Successfully");
                Con.Close();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }
    private void button1_Click(object sender, EventArgs e)
    {
        if (EmpIdCb.SelectedIndex == -1 || PriceTb.Text == "" || ClientNameTb.Text == "" ||
PhoneTb.Text == "" || QuantityTb.Text == "" || TotalTb.Text == "")
        {
            MessageBox.Show("Missing Information");
        }
        else
        {
            try
            {
                Con.Open();
                string Query = "insert into MilkSalesTbl values('" + Date.Value.Date + "',' +
PriceTb.Text + "',' + ClientNameTb.Text + "',' + PhoneTb.Text + "',' +
EmpIdCb.SelectedValue.ToString() + "',' + QuantityTb.Text + "',' + TotalTb.Text + ")";
                SqlCommand cmd = new SqlCommand(Query, Con);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Milk Sold Successfully");

                Con.Close();
                populate();
                SaveTransaction();
                Clear();
            }
            catch (Exception Ex)
            {
                MessageBox.Show(Ex.Message);
            }
        }
    }

    private void button4_Click(object sender, EventArgs e)
    {
        Clear();
    }

    private void pictureBox9_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }

    private void pictureBox10_Click(object sender, EventArgs e)
    {
        Login Ob = new Login();
        Ob.Show();
    }

```

```

        this.Hide();
    }
}

```

Program.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Splash());
        }
    }
}

```

Splash.cs:

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DairyFarmSystem
{
    public partial class Splash : Form
    {
        public Splash()
        {
            InitializeComponent();
        }
        int startpoint = 0;
        private void timer1_Tick(object sender, EventArgs e)
        {
            startpoint += 1;
            Myprogress.Value = startpoint;
            if(Myprogress.Value == 100)
            {
                Myprogress.Value = 0;
                timer1.Stop();
                Login log = new Login();
                this.Hide();
                log.Show();
            }
        }
        private void Splash_Load(object sender, EventArgs e)
        {
            timer1.Start();
        }
    }
}

```

5.2 TESTING APPROACH

5.2.1 UNIT TESTING

Unit testing deals with testing a unit or module as a whole. This would test the interaction of many functions but do confine the test within one module.

In our application we tested each module particularly, to see whether there are any bugs or error while executing the module. Changes were made if any error was found. We went through a pattern of testing which was defined in system testing chapter.

5.2.2 INTEGRATED TESTING

Integrated Testing brings all the modules together into a special testing environment, then checks for errors, bugs and interoperability. It deals with tests for the entire application. After completing unit testing, we combined all the modules together for testing the entire application. Through this technique we came to know about the application limits and features are tested here.

5.3 MODIFICATIONS AND IMPROVEMENTS

- GUI Design Errors removed.

- Made Age Calculation Easier.

- Resolve Database Connectivity Error.

- Cows Module Errors resolved.

Chapter 6

Conclusion:

- This Application can help many dairy farm owners to maintain records with ease and in less time.
- There are modules in this system that helps to analysis the income as well as the expenditure on farm.
- It Can also keep records of Employees, cows and Calves.
- It can provide details of daily milk production and milk sales without any paper work.
- It will the user to take help of technology and go paperless to maintain records.
- It is Easy to Take Backup and Update.

➤ Limitations Of System:

- Sometimes the system takes time to load.
- It may show Errors if not loaded properly.
- One has to be very careful with designing part.

Chapter 7

References

- ❖ Guidance from Prof. Omkar Sherkhane
- ❖ Stackoverflow
- ❖ GeeksforGeeks
- ❖ Youtube
- ❖ ChatGPT
- ❖ Tutorials Point